A Minor Project Report on

EXPLAINABLE AND RELIABLE TRAFFIC-SIGN RECOGNITION ON GTSRB

(ES-451: Minor Project)

submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Information Technology

Submitted by

AKSHIT MEHTA
Enrolment No.: 00314803123

SHASHWAT KUMAR
Enrolment No.: 00414803123

Under the supervision of

Mr. Ajay Kumar Kaushik
Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY
MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY
PSP Area, Plot No. 1, Sector-22, Rohini, Delhi-110086, India

December 2026

DECLARATION

This is to certify that the material embodied in this Minor Project titled "EXPLAINABLE AND RELIABLE TRAFFIC-SIGN RECOGNITION ON GTSRB", being submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is based on our original work. It is further certified that this Minor Project work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. Our indebtedness to other works has been duly acknowledged at the relevant places.

(Akshit Mehta)
Enrolment No.: 00314803123

(Shashwat Kumar)
Enrolment No.: 00414803123

Date: _____
Place: Delhi

Department of Information Technology

CERTIFICATE

This is to certify that the work embodied in this Minor Project titled “EXPLAINABLE AND RELIABLE TRAFFIC-SIGN RECOGNITION ON GTSRB”, being submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is original and has been carried out by Akshit Mehta (Enrolment No. 00314803123) and Shashwat Kumar (Enrolment No. 00414803123) under my supervision and guidance.

It is further certified that this Minor Project work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

Mr. Ajay Kumar Kaushik
Assistant Professor
Department of Information Technology
Maharaja Agrasen Institute of Technology

Prof. (Dr.) Amita Goel
Head, Department of Information Technology
Maharaja Agrasen Institute of Technology

ACKNOWLEDGEMENT

We express our sincere gratitude to our mentor and faculty supervisor, Mr. Ajay Kumar Kaushik, Assistant Professor, Department of Information Technology, Maharaja Agrasen Institute of Technology, for the guidance, encouragement and feedback received throughout this project.

We thank the Project Convenors, Dr. Shallu Bashambu and Mr. Vibhor Sharma, for organising the project reviews and defences and for the clear guidelines that helped us prepare this report. We are grateful to Prof. (Dr.) Amita Goel, Head of the Department of Information Technology, and to the Maharaja Agrasen Institute of Technology for providing an environment in which students are encouraged to carry out independent technical work.

We acknowledge the creators of the German Traffic Sign Recognition Benchmark, whose openly available dataset made this study possible, and the maintainers of the open-source tools on which we relied: TensorFlow and Keras, scikit-learn, NumPy, pandas, Matplotlib, Pillow and OpenCV. We also thank the Google Colaboratory service for the GPU runtime used to train the models.

Finally, we thank our families and friends for their constant encouragement, and our classmates for the many discussions that sharpened our ideas.

(Akshit Mehta)
Enrolment No.: 00314803123

(Shashwat Kumar)
Enrolment No.: 00414803123

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY



VISION OF THE INSTITUTE

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity.

MISSION OF THE INSTITUTE

M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning.

M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.

M3. To digitize educational resources and process for enhanced teaching and effective learning.

M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.

M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.

DEPARTMENT OF INFORMATION TECHNOLOGY

VISION OF THE DEPARTMENT

To establish a center of excellence promoting Information Technology related education and research for preparing technocrats and entrepreneurs with ethical values.

MISSION OF THE DEPARTMENT

M1. To excel in the field by imparting quality education and skills for software development and applications.

M2. To establish a conducive environment that promotes intellectual growth and research.

M3. To facilitate students to acquire entrepreneurial skills for innovation and product development.

M4. To encourage students to participate in competitive events and industry interaction with a focus on continuous learning.

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY



PROGRAM OUTCOMES (POs)

PO1 Engineering knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3 Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4 Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5 Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6 The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7 Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8 Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9 Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10 Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11 Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1 Students are equipped with programming technologies applied in the industry.

PSO2 Students are able to provide efficient solutions for emerging challenges in the computation domain such as Artificial Intelligence, Data Science, Web Technologies.

PSO3 Students are able to apply domain knowledge and expertise in the field of Information Technology for enhancing research capability to transform innovative ideas into reality.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1 The graduates will utilize their professional skills in Information Technology to address real-world and societal challenges.

PEO2 The graduates would become successful entrepreneurs and motivators.

PEO3 The graduates would pursue research globally and engage in producing novel products to serve humanity.

PEO4 The graduates would exhibit collaborative abilities with adherence to professional ethics.

SUSTAINABLE DEVELOPMENT GOALS (SDGs) ADDRESSED

SDG addressed: SDG 9 (Industry, Innovation and Infrastructure); SDG 11 (Sustainable Cities and Communities); SDG 3 (Good Health and Well-being).

How the project contributes towards the above SDGs:

SDG 11 (target 11.2) and SDG 3 (target 3.6). Road traffic injuries are a major cause of death worldwide [1]. Driver-assistance systems that recognise traffic signs reliably can reduce errors caused by missed or misread signs. This project studies, on a public benchmark, how to build such a classifier so that its measured accuracy is honest, its decisions can be inspected, and it can say when it is unsure, which are the properties a safety-relevant component needs before any further validation.

SDG 9 (targets 9.5 and 9.c). The work promotes innovation and access to technology: it delivers a small model (116,203 parameters) that runs entirely in a web browser without a server, so the technique can be examined and reused on low-cost devices and in education without specialised infrastructure. All code, results and the interactive demonstration are released openly.

The project does not claim to be a certified driving system; its limitations are discussed in Chapter 11.

MINOR PROJECT RUBRICS

Maharaja Agrasen Institute of Technology
Department of Information Technology
Project Rubrics

Review 1 – Synopsis (10 Marks)							
Competency	**Performance Indicator**	**Marks**	**20% – Poor**	**40% – Below Average**	**60% – Satisfactory**	**80% – Good**	**100% – Excellent**
Demonstrate an ability to identify and formulate complex engineering problem	Articulate problem statements and identify objectives	3	Problem unclear or irrelevant	Vague statement, little relevance	Reasonably defined, partially relevant	Clear and relevant problem statement	Clear, specific, and highly relevant problem with measurable objectives
	Identify engineering systems, variables, and parameters to solve the problems	3	Minimal sources, no gap	Few sources, unclear gap	Adequate sources, partial gap identification	Good sources, clear gap	Extensive literature, well-defined gap
	Compare and contrast alternative solution processes to select the best process	4	No clear method	Vague method, not feasible	Feasible but lacking detail	Well-detailed and feasible	Highly detailed, justified, innovative method

Review 2 – Mid-term Defence (10 Marks)							
Competency	**Performance Indicator**	**Marks**	**20% – Poor**	**40% – Below Average**	**60% – Satisfactory**	**80% – Good**	**100% – Excellent**
Demonstrate an ability to select an optimal design scheme for further development	Apply formal decision-making tools to select optimal engineering design solutions	4	No analysis	Minimal, flawed analysis	Basic correct analysis	Good analysis with insights	In-depth, accurate, insightful analysis
Demonstrate an ability to generate a diverse set of alternative design solutions	Build models/prototypes to develop a diverse set of design solutions	3	No appropriate tools used	Tools misapplied	Some tools used correctly	Appropriate tools used effectively	Optimal and innovative use of tools
	Able to produce a variety of potential design solutions suited to meet functional requirements	3	No progress	Minimal progress	Partial progress	Mostly complete	Fully complete as per plan

Review 3 – Final Defence (10 Marks)							
Competency	**Performance Indicator**	**Marks**	**20% – Poor**	**40% – Below Average**	**60% – Satisfactory**	**80% – Good**	**100% – Excellent**
Demonstrate an ability to analyze data and reach a valid conclusion	Use appropriate procedures, tools and techniques to conduct experiments and collect data	3	Not implemented	Minimal implementation	Partially functional	Mostly functional	Fully functional as per objectives
	Analyze data for trends and correlations, stating possible errors and limitations	2	No results	Results unclear	Results partially validated	Results validated	Results thoroughly validated with multiple methods
Demonstrate an ability to select and apply discipline specific tools, techniques and resources	Demonstrate proficiency in using discipline-specific tools	3	Poor teamwork	Uneven contribution	Acceptable coordination	Good coordination	Excellent teamwork with equal contribution
Demonstrate an ability to evaluate the suitability and limitations of tools used to solve an engineering problem	Discuss limitations and validate tools, techniques and resources	2	Cannot answer	Answers few questions	Answers most questions	Confident answers	Confident, comprehensive answers to all questions

Review 4 Final Report / Paper (10 Marks)							
Competency	**Performance Indicator**	**Marks**	**20% – Poor**	**40% – Below Average**	**60% – Satisfactory**	**80% – Good**	**100% – Excellent**
Demonstrate an ability to describe engineering roles in a broader context, e.g. pertaining to the environment, health, safety, legal and public welfare	Identify and describe various engineering roles; particularly as pertains to protection of the public and public interest at the global, regional and local level	2	Limited knowledge of public and environment issues	Knowledge of public and environment issues but not clear in thoughts	Moderate understanding of public and environment issues with a view on addressing such issues	Adequate understanding of social contexts and a roadmap of solution	Clear understanding of social issues with a vision of national and global policies
Demonstrate an ability to apply the Code of Ethics	Examine and apply moral & ethical principles to known case studies	2	No originality	Limited originality	Adequate originality	Good originality	Highly original and innovative
Demonstrate an ability to form a team and define a role for each member	Recognize a variety of working and learning preferences; appreciate the value of diversity on a team	2	Poor formatting	Minimal formatting	Adequate formatting	Well-formatted	Professionally formatted, excellent referencing
Demonstrate competence in listening, speaking, and presentation	Deliver effective oral presentations to technical and non-technical audiences	2	No conclusions	Minimal conclusions	Adequate conclusions	Clear conclusions	Strong conclusions with visionary future scope
Demonstrate an ability to identify changing trends in engineering knowledge and practice	Recognize the need and be able to clearly explain why it is vitally important to keep current regarding new developments in your field	2	Very limited: Shows minimal awareness of new trends or developments.	Basic: Demonstrates basic awareness of new developments but explanation is superficial or generic.	Shows moderate understanding of how new developments influence engineering work.	Provides a thoughtful explanation of the importance of keeping updated and links it effectively to professional growth.	Provides a clear, insightful, and well-reasoned explanation of why continuous learning is vital, supported by examples.

ABSTRACT

Reliable recognition of traffic signs is a building block of driver-assistance systems, but published accuracy figures on the German Traffic Sign Recognition Benchmark (GTSRB) say little about how a model behaves on unseen signs, why it errs, or when it should not be trusted. This project builds and evaluates a complete, reproducible traffic-sign classification pipeline that treats these questions as first-class goals.

The 39,209 training images of GTSRB were shown to come from only 1,307 physical signs photographed about 30 times each. A random validation split places 1,291 of those signs on both sides and inflates validation scores, so a per-class, track-grouped split with zero shared signs was used instead. Two models were compared: a compact 116,203-parameter convolutional network trained from scratch and MobileNetV2 transfer learning in two stages. Each design decision was measured on a progress ladder from a trivial baseline (5.9 % validation accuracy) to the final network (97.6 %), a 39-fold reduction in error.

On the official test set, scored once, the custom CNN reached 96.96 % accuracy and 95.21 % macro-F1, ahead of fine-tuned MobileNetV2 (91.26 %) and an ensemble of both (96.88 %). Grad-CAM, calibration (expected calibration error 0.0105), selective prediction (99 % accuracy while answering 95.5 % of images) and corruption tests showed that the network tolerates blur, darkening and glare but is fragile to noise, occlusion and large rotation, and that its most confident errors are very dark speed-limit signs. Two runs of the pipeline differed by about half a point, which bounds what small differences can be trusted.

The trained network was re-implemented in dependency-free JavaScript and verified against Keras (largest probability difference 3.2×10^{-6} over 116 images), and is published as a static website that classifies test images, uploads and camera frames locally, explains each prediction and stress-tests it. The report documents the methodology, implementation, testing, results and limitations, including that the system handles only cropped signs from one country and is not safety-certified.

Keywords: traffic-sign recognition, GTSRB, convolutional neural network, transfer learning, Grad-CAM, calibration, robustness, data leakage.

TABLE OF CONTENTS

Declaration..... ii
Certificate..... iii
Acknowledgement..... iv
Vision and Mission of the Institute and the Department..... v
Program Outcomes, Program Specific Outcomes and Program Educational Objectives..... vi
Sustainable Development Goals (SDGs) Addressed..... viii
Minor Project Rubrics..... ix
Abstract..... xiii
List of Figures..... xviii
List of Tables..... xx

Chapter 1: Introduction..... 1
1.1 Background and Motivation..... 1
1.2 Need of the Study..... 1
1.3 Scope of the Study..... 2
1.4 Objectives of the Study..... 3
1.5 Contributions..... 3
1.6 Organisation of the Report..... 4

Chapter 2: Problem Statement..... 6
2.1 Problem Definition..... 6
2.2 Objectives..... 6
2.3 Research Questions..... 7
2.4 Evaluation Protocol..... 7
2.5 Success Criteria..... 7

Chapter 3: Literature Review..... 9
3.1 Traffic-Sign Recognition and the GTSRB Benchmark..... 9
3.2 Convolutional Networks and Their Components..... 9
3.3 Transfer Learning and MobileNetV2..... 10
3.4 Evaluation Pitfalls: Leakage, Dataset Bias and Imbalance..... 10
3.5 Explainability: Grad-CAM and Embedding Visualisation..... 11
3.6 Calibration and Selective Prediction..... 11
3.7 Robustness to Corruptions..... 11
3.8 Detection, Datasets and Safety Standards..... 12
3.9 Summary and Research Gap..... 12

Chapter 4: Analysis..... 14

4.1 Software Requirement Specifications.....14
4.1.1 Functional Requirements of the Project.....14
4.1.2 Non-functional Requirements of the Project.....15
4.2 Feasibility Study of the Project.....15
4.3 Tools, Technologies and Platform Used.....16
4.4 Use Case Diagrams and Data Flow Diagrams.....17
Chapter 5: Research Methodology.....20
5.1 Methodology Used.....20
5.1.1 Principles.....20
5.1.2 Evaluation Metrics.....20
5.1.3 Experimental Design.....21
5.2 Data Collection and Analysis.....22
5.2.1 Dataset and Acquisition.....22
5.2.2 Exploratory Data Analysis.....22
5.2.3 Leakage Analysis.....24
5.3 Tools and Technology Used.....25
Chapter 6: Design and Architecture.....26
6.1 Structure Chart and Work Breakdown Structure.....26
6.2 Explanation of Modules.....27
6.3 Flow Chart and Activity Diagram.....27
6.4 ER Diagram and Class Diagram.....29
6.5 Model Architectures and Algorithms.....30
6.5.1 Custom CNN.....30
6.5.2 MobileNetV2 Transfer Learning.....31
6.5.3 Algorithms.....32
Chapter 7: Implementation.....34
7.1 Data Presentation and Preparation.....34
7.1.1 Loading and Verification.....34
7.1.2 The Validation Split.....34
7.1.3 Caching and Augmentation.....35
7.1.4 Hyper-parameters.....36
7.2 Implementation of the Work.....36
7.2.1 Models and Training.....36
7.2.2 The Progress Ladder.....37
7.2.3 Evaluation, Explanation and Reliability Code.....37
7.2.4 Export and Parity Checks.....37
7.2.5 The Browser Implementation.....38

7.3 Analysis and Interpretation of Training....38
7.4 Screenshots....40
7.5 Source Code of Some Modules....42
Chapter 8: Testing....44
8.1 Test Strategy....44
8.2 Checks Embedded in the Pipeline....44
8.3 Numerical Verification of the Browser Network....45
8.4 Functional Test Cases for the Website....45
8.5 Defects Found and Fixed....46
8.6 Limitations of the Testing....47
Chapter 9: Results and Discussion....48
9.1 Results....48
9.1.1 Model Comparison....48
9.1.2 Progress Ladder....49
9.1.3 Per-Class Performance....50
9.1.4 Confusion Analysis....51
9.1.5 Explainability....52
9.1.6 Calibration and Selective Prediction....53
9.1.7 Robustness to Corruptions....54
9.1.8 Export and Deployment....55
9.2 Discussion....56
9.2.1 Merits and Demerits of the Approach....56
9.2.2 Comparison with Published Results....56
9.2.3 Why Did the Small CNN Beat MobileNetV2?....56
9.2.4 Grad-CAM at Saturation....57
9.2.5 Run-to-Run Variation....57
9.3 Findings....58
Chapter 10: Summary and Conclusion....60
10.1 Summary of the Work....60
10.2 Conclusions....61
10.3 Learning Outcomes....61
Chapter 11: Limitations of the Project and Future Work....62
11.1 Limitations....62
11.2 Future Work....63
11.3 Ethical and Safety Considerations....63
References....64
Appendix A: Source Code of Selected Modules (Annexure 1)....67

Appendix B: Per-Class Results of the Custom CNN.....69

Appendix C: Reproducing the Work.....71

C.1 Running the Experiments.....71

C.2 Rebuilding the Website Data.....71

C.3 Repository Layout.....71

Appendix D: Research and Publication Status (Annexure 2).....73

LIST OF FIGURES

Figure No. and Title **Page No.**

Figure 4.1: Use case diagram of the traffic-sign recognition system....17
Figure 4.2: Data flow diagram, level 0 (context diagram)....18
Figure 4.3: Data flow diagram, level 1....19
Figure 5.1: End-to-end workflow of the study....20
Figure 5.2: Number of images per class in the training and official test sets....23
Figure 5.3: One random training example per class....23
Figure 5.4: Distribution of image width, size and aspect ratio in the training images....24
Figure 5.5: Brightness distribution of the training images with the darkest and brightest examples....24
Figure 5.6: Frames of a single physical Stop sign (one training track)....24
Figure 5.7: Random image split versus the track-grouped split used in this study....25
Figure 6.1: Work breakdown structure of the project....26
Figure 6.2: Flow chart of the experiment pipeline....28
Figure 6.3: Data and control flow of the website....28
Figure 6.4: Entity-relationship diagram of the data and results....29
Figure 6.5: Class and module diagram of the code....30
Figure 6.6: Architecture of the custom CNN....31
Figure 6.7: Two-stage training of the MobileNetV2 model....32
Figure 7.1: Original image and random augmentations (training only)....36
Figure 7.2: Training and validation curves of the custom CNN....39
Figure 7.3: Training and validation curves of MobileNetV2 across the two stages (the dashed line marks the start of fine-tuning)....39
Figure 7.4: Website home view with the live prediction panel and headline results....40
Figure 7.5: Live demonstration of a misclassified test image with Grad-CAM....41
Figure 7.6: Noise applied to a test image; the model would defer....41
Figure 7.7: Stress lab with presets, sliders and a noise sweep....42
Figure 7.8: Results dashboard: the progress ladder and training curves....42
Figure 9.1: Test accuracy and macro-F1 of the models, and accuracy versus latency....49
Figure 9.2: Progress ladder: validation accuracy at each step (left) and the same data as error rate on a logarithmic scale (right)....49
Figure 9.3: F1 score of every class (left) and F1 against class size (right)....51
Figure 9.4: Row-normalised confusion matrix of the custom CNN on the test set....52
Figure 9.5: The twelve most confident errors on the test set....52
Figure 9.6: Grad-CAM for the custom CNN (middle row) and MobileNetV2 (bottom row): five correct predictions (left) and five confident errors (right)....53
Figure 9.7: t-SNE projection of the penultimate-layer embedding, coloured by sign family and by class....53
Figure 9.8: Confidence of correct and wrong predictions, reliability diagram and coverage-accuracy curve....54

Figure 9.9: Accuracy of the custom CNN under increasing corruption severity.....55
Figure 9.10: Examples of the corruptions at the three severities.....55

LIST OF TABLES

Table No. and Title **Page No.**

Table 1.1: Mapping of the required report headings to the chapters of this report....4
Table 2.1: Objectives and the evidence that shows each has been met....6
Table 3.1: Summary of the reviewed work and its relevance to this project....12
Table 4.1: Functional requirements....14
Table 4.2: Non-functional requirements....15
Table 4.3: Feasibility analysis....16
Table 4.4: Tools, technologies and platform....16
Table 4.5: Principal use cases....17
Table 5.1: Training budgets of the notebook run modes (epochs are upper limits)....22
Table 5.2: Dataset facts measured by the notebook....22
Table 6.1: Work packages and deliverables....26
Table 6.2: Modules and their responsibilities....27
Table 6.3: Layers and parameters of the custom CNN....31
Table 7.1: Random split versus the track-grouped split used....35
Table 7.2: Augmentation settings (training only)....35
Table 7.3: Hyper-parameters and training settings of the reported run....36
Table 7.4: Definition of the progress-ladder steps....37
Table 7.5: Training cost of the reported run....39
Table 8.1: Levels of testing used....44
Table 8.2: Automated checks in the notebook and their outcome in the reported run....44
Table 8.3: Agreement between the JavaScript network and Keras on 116 test images....45
Table 8.4: Functional test cases for the website....45
Table 8.5: Defects found during development and testing....46
Table 9.1: Performance of the four models on the official test set (12,630 images)....48
Table 9.2: Progress ladder: validation accuracy and macro-F1 at each step....49
Table 9.3: The eight classes with the lowest F1 on the test set....50
Table 9.4: The ten most frequent confusions on the test set....51
Table 9.5: Selective prediction of the custom CNN on the test set....54
Table 9.6: Accuracy of the custom CNN under corruptions (6,000 test images; clean accuracy 96.97 %)....54
Table 9.7: Merits and demerits of the proposed approach and pipeline....56
Table 9.8: Test accuracy in the two full runs....57
Table 10.1: Answers to the research questions....61
Table 11.1: Proposed future work and the evidence that motivates it....63
Table B.1: Per-class test results of the custom CNN....69
Table C.1: Layout of the project repository....71
Table D.1: Research and publication status....73

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Road traffic crashes remain one of the leading causes of death and injury worldwide [1]. Many crashes involve a driver who failed to notice, or misread, a traffic sign: a speed limit passed unseen, a stop sign hidden by glare, a warning obscured by vegetation. Advanced driver-assistance systems (ADAS) and, in the longer term, automated vehicles therefore need a component that recognises traffic signs automatically and reliably.

Recognising a traffic sign from a camera image is a classic image-classification problem. It looks easy for a person, but it is demanding for a machine. Signs are photographed at different distances, so the same sign may occupy fewer than thirty or more than two hundred pixels. Lighting ranges from direct sun to dusk. Signs are tilted, dirty, partly covered, motion-blurred and surrounded by clutter. Many signs also differ only in a small detail: a speed limit of 60 km/h and one of 80 km/h share the same shape, colour and layout and differ by a single digit.

Deep convolutional neural networks (CNNs) have made this problem largely tractable. On the German Traffic Sign Recognition Benchmark (GTSRB), the dataset used in this project, published committees of CNNs exceed the measured accuracy of human participants [2], [3]. A reader might therefore conclude that traffic-sign classification is a solved problem. This project starts from a different observation: a high number on a benchmark leaves several practical questions unanswered. Was the number measured honestly? On which signs does the model fail, and why? Does it look at the sign or at the background? Does it know when it is unsure? What happens when the image is darker, noisier or partly covered than anything seen in training? And can the result be reproduced and inspected by someone else?

1.2 Need of the Study

The study is motivated by five gaps between benchmark accuracy and trustworthy deployment.

- **Evaluation can be optimistic through data leakage.** Leakage occurs when information from the evaluation data reaches the training process [4]. GTSRB training images are frames taken from short sequences of the same physical

sign. If a validation set is drawn at random from these frames, near-duplicates of training images appear in it and validation accuracy overstates performance on new signs. Whether this matters for GTSRB, and by how much, should be measured rather than assumed.

- **Accuracy alone hides class-level behaviour.** The 43 classes are strongly imbalanced (about 10.7 times between the largest and the smallest class). Overall accuracy can be high while rare classes fail, and some confusions (a wrong speed limit) are far costlier than others. Per-class metrics, macro-averaged F1 and a confusion analysis are needed.
- **Black-box predictions are hard to trust.** A classifier that is right for the wrong reasons, for example by reading background cues, will fail unpredictably. Visual explanation methods such as Grad-CAM [5] provide evidence about what the network attends to, but they must themselves be checked for numerical reliability.
- **Confidence must be meaningful.** A safety-relevant component should know when to defer. Modern networks are frequently over-confident [6], and a reject option can trade coverage for accuracy [7]. Both need to be quantified.
- **Robustness and reproducibility are rarely reported together with accuracy.** Corruptions such as noise, blur and occlusion expose weaknesses that clean test accuracy hides [8]. Finally, results that others cannot reproduce, or that stop at a notebook, have limited value.

1.3 Scope of the Study

In scope. The project covers classification of cropped, single traffic-sign images into the 43 GTSRB classes. It includes verified acquisition of the dataset; exploratory analysis; a leakage-free validation design; a custom CNN trained from scratch and a MobileNetV2 transfer-learning model; a ladder of experiments from a naive baseline to the final model; evaluation on the official test set; per-class and confusion analysis; Grad-CAM and embedding visualisation; calibration, selective prediction and robustness tests; export of models with parity checks; and a static website in which the trained network runs entirely in the browser.

Out of scope. Traffic-sign *detection* in full scenes and video (this needs a detector and box annotations that GTSRB does not provide [9], [10]), signs from other countries,

adversarial attacks, hardware benchmarking on embedded automotive processors, and any claim of functional-safety certification [11], [12].

1.4 Objectives of the Study

1. To acquire and verify the GTSRB dataset reproducibly and to characterise its class balance, image size, lighting and internal structure.
2. To design a validation procedure that is free of leakage and to quantify the leakage that a random split would cause.
3. To train and compare a compact CNN and a transfer-learning model, and to measure the contribution of each design decision through a progress ladder.
4. To evaluate the final models once on the official test set using accuracy, macro-F1, top-3 accuracy and calibration error, and to analyse per-class results and confusions.
5. To explain predictions with Grad-CAM and embedding visualisation and to check the numerical reliability of the explanation.
6. To measure calibration, selective prediction and robustness to controlled corruptions.
7. To export the trained models with verified reload and conversion parity, and to deliver an interactive, serverless demonstration of the selected model.
8. To document the work, its limitations and its reproducibility honestly, including run-to-run variation.

1.5 Contributions

The main contributions of this work are the following.

- A measured demonstration that a random validation split on GTSRB places 1,291 of 1,307 physical signs on both sides of the split, together with a per-class, track-grouped split that guarantees zero shared signs and presence of all 43 classes in validation.
- A progress ladder that raises validation accuracy from 5.9 % (majority-class baseline) to 97.6 % in single-factor steps, showing that a 116,203-parameter CNN with batch normalisation, global average pooling, augmentation and dropout outperforms both a 1.63-million-parameter plain CNN and a fine-tuned 2.3-million-parameter MobileNetV2.

- A reliability analysis: expected calibration error of 0.0105, a threshold that gives 99 % accuracy while answering 95.5 % of images, and a corruption study that identifies noise, occlusion and large rotation as the dominant weaknesses.
- A numerical finding about Grad-CAM: when the class probability is used as the score, the gradient vanishes for nearly certain predictions and the maps are unreliable; the class score before the softmax avoids this.
- A dependency-free JavaScript implementation of the trained network, with Grad-CAM, verified against Keras and published as a static website; the same code base also exports all trained models and their results for reuse.

1.6 Organisation of the Report

The department's guidelines (Annexure A) define separate chapter structures for development-based and for research-based projects. This project has both a research component (an experimental study of evaluation and reliability) and a development component (a deployable pipeline and website), so this report follows a combined structure that contains every heading required by either format. Table 1.1 shows where each required heading is located.

Table 1.1: Mapping of the required report headings to the chapters of this report

Required heading (Annexure A)	Format	Location in this report
Introduction	Both	Chapter 1
Problem Definition; Objectives	Both	Chapter 2 (2.1, 2.2)
Literature Review	Research	Chapter 3
Software Requirement Specifications (functional and non-functional requirements)	Development	Chapter 4 (4.1)
Feasibility Study	Development	Chapter 4 (4.2)
Tools / Technologies / Platform Used	Both	Chapter 4 (4.3); 5.3
Use Case Diagrams / Data Flow Diagrams	Development	Chapter 4 (4.4)
Methodology Used	Research	Chapter 5 (5.1)
Data Collection and Analysis	Research	Chapter 5 (5.2)
Structure Chart / Work Breakdown Structure; Explanation of Modules	Development	Chapter 6 (6.1, 6.2)
Flow Chart / Activity Diagram; ER Diagram / Class Diagram; Algorithm	Development	Chapter 6 (6.3 to 6.5)
Data Presentation; Implementation of the Work; Analysis and Interpretation	Research	Chapter 7 (7.1 to 7.3)
Screenshots; Source Code of Some Modules	Development	Chapter 7 (7.4, 7.5); Appendix A

Required heading (Annexure A)	Format	Location in this report
Testing (including test cases)	Development	Chapter 8
Results; Discussion; Findings (merits and demerits)	Research	Chapter 9
Summary and Conclusion	Both	Chapter 10
Limitations of the Project and Future Work	Both	Chapter 11
Bibliography; Appendix	Both	References; Appendices A to C

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

Let *x* denote a colour image of a single traffic sign cropped from a road scene, represented as an array of 64 x 64 x 3 pixel values in the range 0 to 255. Let *y* be the class label of the sign, an integer from 0 to 42, corresponding to the 43 sign types of GTSRB (for example, class 14 is “Stop” and class 2 is “Speed limit 50 km/h”). The task is to learn a function *f* that maps *x* to a probability distribution over the 43 classes, from which the predicted class and a confidence value are obtained:

$$p(y = c \mid x) = \exp(z_c) / \Sigma_j \exp(z_j), \quad \hat{y} = \operatorname{argmax}_c \, p(y = c \mid x) \qquad (1)$$

where *z* is the vector of 43 scores (logits) produced by the network. The problem studied in this report is broader than fitting *f*. It is to obtain, for such a classifier, an **honest estimate of its accuracy on signs it has never seen**, an **explanation** of its predictions, a measure of **when its confidence can be trusted**, a measure of **how it degrades** under realistic image corruptions, and a **reproducible, deployable artefact**.

The question can be stated compactly: *given the structure of GTSRB, how should a traffic-sign classifier be trained, evaluated, explained and delivered so that its reported performance is trustworthy and its limitations are known?*

2.2 Objectives

The objectives listed in Section 1.4 are made measurable below (Table 2.1).

Table 2.1: Objectives and the evidence that shows each has been met

No.	Objective	Evidence in this report
O1	Reproducible, verified data	MD5 checksum and count assertions (Section 5.2)
O2	Leakage-free validation	Zero shared tracks, all 43 classes in validation (Section 7.1.2)
O3	Compare models; measure each design step	Progress ladder (Section 9.1.2)
O4	Honest final evaluation	Single official-test evaluation; metrics and per-class analysis (Chapter 9)
O5	Explain predictions	Grad-CAM, embeddings, numerical check (Sections 9.1.5, 9.2.4)
O6	Reliability	Calibration, abstention, robustness (Sections 9.1.6, 9.1.7)
O7	Deployable and verified	Export, TFLite, Keras-to-JavaScript parity (Sections 8.3, 9.1.8)

No.	Objective	Evidence in this report
O8	Honest documentation	Run-to-run variation and limitations (Section 9.2.5, Chapter 11)

2.3 Research Questions

- **RQ1.** How much does a random validation split overstate performance on GTSRB, and how can the leakage be removed?
- **RQ2.** How do a small purpose-built CNN and a pretrained MobileNetV2 compare on 64 x 64 sign crops, and which design decisions contribute most?
- **RQ3.** Which classes and class pairs are hardest, and are the errors explained by class size or by visual similarity?
- **RQ4.** Does the network attend to the sign, and is Grad-CAM numerically reliable for confident predictions?
- **RQ5.** Is the confidence well calibrated, and how much accuracy can be gained by abstaining when unsure?
- **RQ6.** Which corruptions degrade the model most, and what does this imply for future training?

2.4 Evaluation Protocol

The evaluation protocol was fixed before results were examined, to avoid choosing a protocol that flatters the results. Training data are split into a fitting part and a validation part using whole physical signs. All decisions (when to stop, which model to select, whether a design step helps) use validation data only. The official test set is used exactly once for each model, for reporting. Because the classes are imbalanced, macro-averaged F1 is reported alongside accuracy. Descriptive analyses that use test-set behaviour (the abstention curve and the robustness study) are labelled as such and are not used to choose a model.

2.5 Success Criteria

- The validation split shares no physical sign with the fitting split and contains every class (checked by assertions in the code).
- The selected model reaches test accuracy in the range reported for compact CNNs on GTSRB, without any use of the test set for tuning.

- Every exported model reloads with identical predictions, and the browser implementation reproduces Keras predictions on the sample images.
- All limitations that affect interpretation of the results are stated in the report.

CHAPTER 3: LITERATURE REVIEW

This chapter reviews the work on which the project builds: traffic-sign recognition and the GTSRB benchmark, the building blocks of convolutional networks, transfer learning, evaluation pitfalls, explainability, calibration and robustness. It ends with the gap that this project addresses. A summary table is given at the end (Table 3.1).

3.1 Traffic-Sign Recognition and the GTSRB Benchmark

Early traffic-sign recognition systems relied on hand-crafted features such as colour thresholds, shape detection and histogram-of-gradients descriptors combined with classifiers such as support vector machines. These were sensitive to lighting and viewpoint. The German Traffic Sign Recognition Benchmark, introduced for a competition at the International Joint Conference on Neural Networks in 2011, provided a large, realistic and fixed benchmark: more than 50,000 images in 43 classes, collected on German roads under varied conditions, with a prescribed training set and a separate test set [13]. The accompanying study compared machine-learning methods with human participants and reported human performance of about 98.8 % on the test set [2].

Convolutional networks dominated the competition. Sermanet and LeCun used multi-scale convolutional networks that feed features from several stages into the classifier [14]. Cireşan et al. combined many deep networks, trained on differently pre-processed inputs, into a multi-column committee that exceeded human accuracy, reaching about 99.5 % [3]. These results established that CNNs are well suited to the task. They also show that the accuracies of strong methods are very close to each other, so that how a model is evaluated matters as much as which model is used.

3.2 Convolutional Networks and Their Components

A convolutional network applies learned filters across an image, sharing weights across positions, and stacks such layers so that early layers detect edges and later layers detect shapes and symbols. LeCun et al. showed the effectiveness of this design for document recognition [15], and Krizhevsky et al. demonstrated its power on large-scale natural images [16]. A textbook treatment is given by Goodfellow et al. [17]. Several components are used in this project:

- **Rectified linear units (ReLU)**, defined as max(0, x), make training fast and avoid saturation [18].
- **Batch normalisation** normalises layer activations using batch statistics during training and stored running statistics at inference, which stabilises and accelerates training [19].
- **Dropout** randomly disables units during training to reduce over-fitting [20].
- **Adam** is an adaptive-rate optimiser based on estimates of the first and second moments of the gradient [21].
- **Residual connections** allow very deep networks to be trained [22]; they appear inside the MobileNetV2 backbone used here.

3.3 Transfer Learning and MobileNetV2

Transfer learning reuses a model trained on a large source task to solve a target task with less data or time [23]. Yosinski et al. showed that early-layer features are general while later layers are specific, and that transferability decreases as source and target tasks differ [24]. The usual source task is ImageNet classification [25]. A common practice is to train a new output layer on frozen pretrained features and then to fine-tune later layers at a lower learning rate.

MobileNets are efficient networks designed for mobile devices [26]. MobileNetV2 introduces inverted residual blocks with linear bottlenecks and uses depthwise separable convolutions to reduce computation [27]. It is a natural lightweight comparison model for a deployment-oriented study. Its pretrained features come from natural photographs of everyday objects at a few hundred pixels, which differ in scale and content from 64-pixel traffic-sign crops. This domain gap suggests that a frozen backbone may perform poorly and that fine-tuning may help; this project tests that expectation rather than assuming it.

3.4 Evaluation Pitfalls: Leakage, Dataset Bias and Imbalance

Kaufman et al. define leakage as the presence in training data of information that would not be available when the model is used, and show that it can make results look better than they are [4]. Torralba and Efros show that benchmark datasets have characteristic biases that make performance measured on one dataset a poor predictor of performance elsewhere [28]. For GTSRB, the structure of the data is relevant: images come from

tracks of about 30 frames of the same physical sign. A split that separates the frames of a track places near-duplicates on both sides. The standard remedy is a grouped split, available as `GroupShuffleSplit` in scikit-learn [29], [30]. A plain grouped shuffle does not guarantee that every class appears in the held-out part, a limitation that this project met in practice and solved with an explicit per-class selection (Section 7.1).

With imbalanced classes, overall accuracy is dominated by the common classes. Per-class precision and recall, and their harmonic mean F1, give a complete picture; macro-F1 averages F1 over classes with equal weight, so that rare classes count as much as common ones.

3.5 Explainability: Grad-CAM and Embedding Visualisation

Grad-CAM produces a coarse heat-map showing which regions of an image most increased the score of a class. It weights the feature maps of the last convolutional layer by the average gradient of the class score with respect to each map, sums them and keeps the positive part [5]. It requires no change to the network. Its limitations are well known: it shows influence rather than causal reasoning, and its resolution is that of the last feature map. A second tool, t-SNE, projects high-dimensional feature vectors to two dimensions so that similar images appear close together [31]; its distances should be read qualitatively. This project adds a numerical check that is rarely reported: the behaviour of the gradient when the network is almost certain (Section 9.2.4).

3.6 Calibration and Selective Prediction

A classifier is calibrated if, among predictions made with confidence c, a fraction c is correct. Guo et al. showed that modern deep networks are often over-confident, introduced the expected calibration error (ECE) as a summary, and proposed temperature scaling as a simple remedy [6]. Selective classification lets a model abstain when its confidence is below a threshold, trading coverage for accuracy [7]. For a safety-relevant component, abstention is a practical way to hand uncertain cases to a human or another sensor.

3.7 Robustness to Corruptions

Hendrycks and Dietterich proposed benchmarks of common corruptions such as noise, blur and weather to measure how classifiers degrade when inputs differ from the training distribution [8]. More severe is the possibility of deliberate manipulation: small

perturbations can change a network's output [32], and physical stickers on a stop sign have been shown to fool classifiers [33]. This project studies benign corruptions only (darkening, glare, noise, blur, rotation and occlusion) and does not evaluate adversarial attacks.

3.8 Detection, Datasets and Safety Standards

Classification of cropped signs is the second stage of a larger pipeline in which a detector first locates signs in a scene. The German Traffic Sign Detection Benchmark provides annotated full images [9], the Tsinghua-Tencent 100K dataset provides large-scale Chinese traffic-sign data [10], and one-stage detectors such as YOLO can localise signs in real time [34]. Functional safety of road vehicles is governed by ISO 26262 [11], and the safety of the intended functionality of perception components by ISO 21448 [12]. Neither standard is satisfied by a research classifier; they are cited to place the work in context.

3.9 Summary and Research Gap

Table 3.1: Summary of the reviewed work and its relevance to this project

Work	Contribution	Use in this project
Stallkamp et al. [2], [13]	GTSRB benchmark; human performance of about 98.8 %	Dataset, protocol, reference point
Cireşan et al.; Sermanet and LeCun [3], [14]	CNN committees and multi-scale CNNs exceed human accuracy	Motivates a CNN approach; context for accuracy
Ioffe and Szegedy; Srivastava et al. [19], [20]	Batch normalisation; dropout	Components of the custom CNN
Sandler et al.; Yosinski et al. [24], [27]	MobileNetV2; transferability of features	Transfer-learning model; domain-gap hypothesis
Kaufman et al.; scikit-learn [4], [29]	Leakage; grouped splitting	Track-grouped validation design
Selvaraju et al.; van der Maaten and Hinton [5], [31]	Grad-CAM; t-SNE	Explanation of predictions
Guo et al.; Geifman and El-Yaniv [6], [7]	Calibration, ECE; selective classification	Reliability analysis; abstention
Hendrycks and Dietterich [8]	Corruption robustness benchmarks	Stress tests

Research gap. The literature establishes that CNNs solve GTSRB at the level of human accuracy. What is less often reported together, for a single reproducible pipeline, is (i) a measured account of validation leakage caused by the track structure of the data, (ii) a controlled ladder of design decisions from a trivial baseline, (iii) calibration,

abstention and corruption robustness measured on the same model, (iv) a numerical check of the explanation method, and (v) a deployable artefact whose faithfulness to the trained model has been verified. This project addresses these five points.

CHAPTER 4: ANALYSIS

This chapter analyses what the system must do and whether it can be built. It follows the structure required for development-based projects: software requirements, feasibility, tools and platform, and use case and data flow diagrams.

4.1 Software Requirement Specifications

Purpose and scope. The system consists of two cooperating parts. The first is an *experiment pipeline*, run as a notebook on a GPU runtime, that acquires and verifies GTSRB, trains and evaluates models, analyses their behaviour and exports models and results. The second is a *static website* that runs the selected model in a visitor's browser and presents the results. The intended users are the researchers (the project team) who run and extend the pipeline, and examiners or visitors who explore the demonstration.

Assumptions and constraints. The pipeline assumes an internet connection to download the dataset and pretrained weights and a GPU runtime for full-length training. The website assumes a modern browser with JavaScript and canvas support; camera input additionally requires a secure (HTTPS) context and user permission. The model accepts one cropped sign at a time. No server component is used.

4.1.1 Functional Requirements of the Project

Table 4.1: Functional requirements

ID	Requirement	Part
FR1	Download the GTSRB archive and verify it against the published MD5 checksum; stop with an error on mismatch.	Pipeline
FR2	Assert the expected counts (39,209 training images, 12,630 test images, 43 classes) and that every referenced file exists.	Pipeline
FR3	Profile the data: class distribution, image size, brightness, and the track structure of the training images.	Pipeline
FR4	Create a validation split that shares no physical sign with the fitting split and contains every class.	Pipeline
FR5	Cache all images at 64 x 64 resolution and apply training-time augmentation inside the model.	Pipeline
FR6	Train a custom CNN and a two-stage MobileNetV2 with early stopping and learning-rate reduction.	Pipeline
FR7	Run a ladder of single-factor experiments from a trivial baseline to the final model.	Pipeline
FR8	Evaluate each model on the official test set and report accuracy, macro-F1, top-3 accuracy, calibration error, size and latency.	Pipeline

ID	Requirement	Part
FR9	Report per-class metrics, the confusion matrix, the most frequent confusions and the most confident errors.	Pipeline
FR10	Generate Grad-CAM heat-maps and an embedding projection.	Pipeline
FR11	Measure calibration, the accuracy-versus-coverage trade-off and robustness to six corruption types at three severities.	Pipeline
FR12	Export every trained model, per-model predictions and metrics, figures and a sample of test images; verify that reloaded models reproduce predictions.	Pipeline
FR13	Classify a selected test image, an uploaded image or a camera frame in the browser and show the top five classes with confidences.	Website
FR14	Show a Grad-CAM heat-map for the current prediction.	Website
FR15	Let the user apply rotation, brightness, blur, noise and occlusion to the current image and sweep each across its range.	Website
FR16	Let the user set a confidence threshold below which the system defers, and show the resulting coverage and accuracy.	Website
FR17	Present the experiment results as interactive charts with table alternatives.	Website
FR18	Provide a check that compares the browser model with the values recorded from the Keras model.	Website

4.1.2 Non-functional Requirements of the Project

Table 4.2: Non-functional requirements

Category	Requirement
Reproducibility	All random generators are seeded; the dataset is verified by checksum; one configuration object controls budgets; results are exported with the code that produced them.
Correctness of evaluation	No decision may use the test set; validation may not share physical signs with training; assertions enforce this.
Performance	A prediction on the website should complete in well under a second on an ordinary laptop; full training should complete in under one hour on a free GPU runtime.
Portability	The pipeline runs unchanged on Google Colab and, with a local dataset path, on a workstation; the website runs on any modern browser.
Privacy	Images supplied by a website visitor are processed locally and never transmitted.
Usability and accessibility	The website works with keyboard navigation, provides table views for charts, supports light and dark themes and a narrow (phone) layout.
Maintainability	The notebook and website data are generated by scripts; the browser network is covered by a numerical verification script.
Honesty and safety messaging	The system must state that it handles cropped signs only, is not safety-certified and may be over-confident outside its training distribution.

4.2 Feasibility Study of the Project

Table 4.3 summarises the feasibility analysis along the usual dimensions.

Table 4.3: Feasibility analysis

Dimension	Assessment	Conclusion
Technical	GTSRB is small enough (about 320 MB) to fit in memory at 64 x 64 resolution (0.64 GB). A 116,203-parameter CNN trains in about six minutes on a free GPU. A network of this size can be evaluated in plain JavaScript in tens of milliseconds.	Feasible
Economic	All software is open source. Training used a free Colab runtime; hosting uses GitHub Pages. No licences or hardware purchases were needed.	Feasible at near-zero cost
Operational	The pipeline runs by selecting a mode and running all notebook cells; the website needs no installation and no server administration.	Feasible
Schedule	The work packages (Chapter 6) were small and largely sequential; a full training run takes under 25 minutes, which allowed several complete iterations, including a correction of the validation split.	Feasible within the semester
Legal and ethical	GTSRB is a public research benchmark and is cited as required. The website processes images locally. Safety limitations are stated prominently. Redistribution of the cropped sample images should respect the dataset's terms.	Feasible with the stated precautions

4.3 Tools, Technologies and Platform Used

Table 4.4: Tools, technologies and platform

Tool / platform	Version or detail	Purpose
Python	3.x (Colab runtime)	Pipeline language
TensorFlow / Keras [35], [36]	2.21.0 / 3.13.2 (Colab run); pinned 2.16.2 in the repository	Model definition, training, export
NumPy, pandas [37]	NumPy 2.x; pandas 2.x	Arrays, tables
scikit-learn [30]	1.7.x	Metrics, t-SNE, splitting utilities
Matplotlib [38]	3.10.x	Figures
Pillow, OpenCV	Pillow 11.x; OpenCV headless 4.12	Image loading, corruptions
Google Colaboratory [39]	GPU runtime	Training and evaluation
TensorFlow Lite	Bundled with TensorFlow	Mobile-size model conversion
HTML, CSS, JavaScript (ES modules)	No third-party libraries	Website, in-browser network, charts
Node.js	Recent LTS	Numerical verification script
Git and GitHub	-	Version control, collaboration
GitHub Pages and Actions	-	Static hosting and deployment
GTSRB archive [40]	Zenodo record 13741936	Dataset

4.4 Use Case Diagrams and Data Flow Diagrams

Figure 4.1 shows the use cases of the system. The *researcher* runs the experiment pipeline; the *panel member or website visitor* uses the website. Table 4.5 describes the principal use cases.

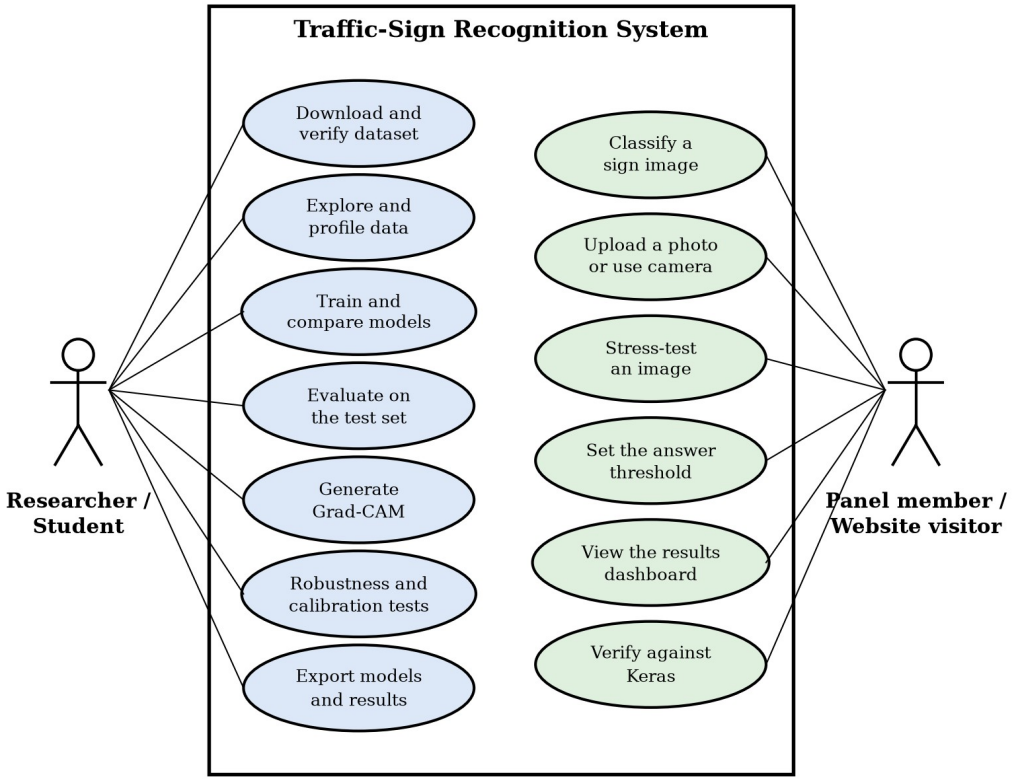


Figure 4.1: Use case diagram of the traffic-sign recognition system

Table 4.5: Principal use cases

Use case	Actor	Main flow	Outcome
Train and compare models	Researcher	Select a mode; run the notebook; models are trained, early stopping selects the best epoch, results are scored on validation	Trained models and training histories
Evaluate on the test set	Researcher	Score each model once on the official test split; compute metrics and analyses	Metrics, confusion matrix, figures
Export results	Researcher	Save models, predictions, tables and figures; reload each model and compare predictions	Verified results bundle
Classify a sign image	Visitor	Pick a sample, upload a photo or use the camera; the model returns the top five classes	Prediction, confidence, answer-or-defer decision
Stress-test an image	Visitor	Move the corruption sliders or press Sweep; the prediction updates	Confidence versus severity
Verify against Keras	Visitor	Press the verify button; the page compares its predictions with recorded Keras values	Agreement report

The context-level data flow diagram (Figure 4.2) shows the system as one process with four external entities: the GTSRB archive, the researcher, the website visitor and the hosting platform. Figure 4.3 decomposes it into five processes (prepare data, train models, evaluate and analyse, export artefacts, browser inference) and five data stores.

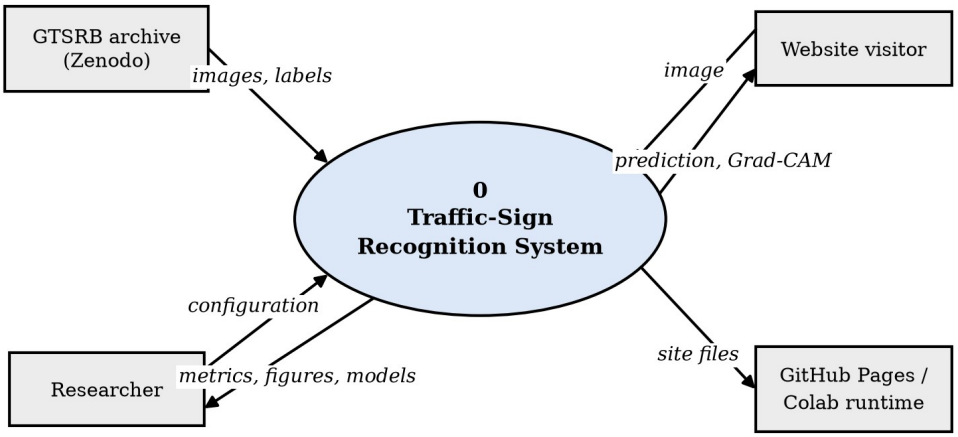


Figure 4.2: Data flow diagram, level 0 (context diagram)

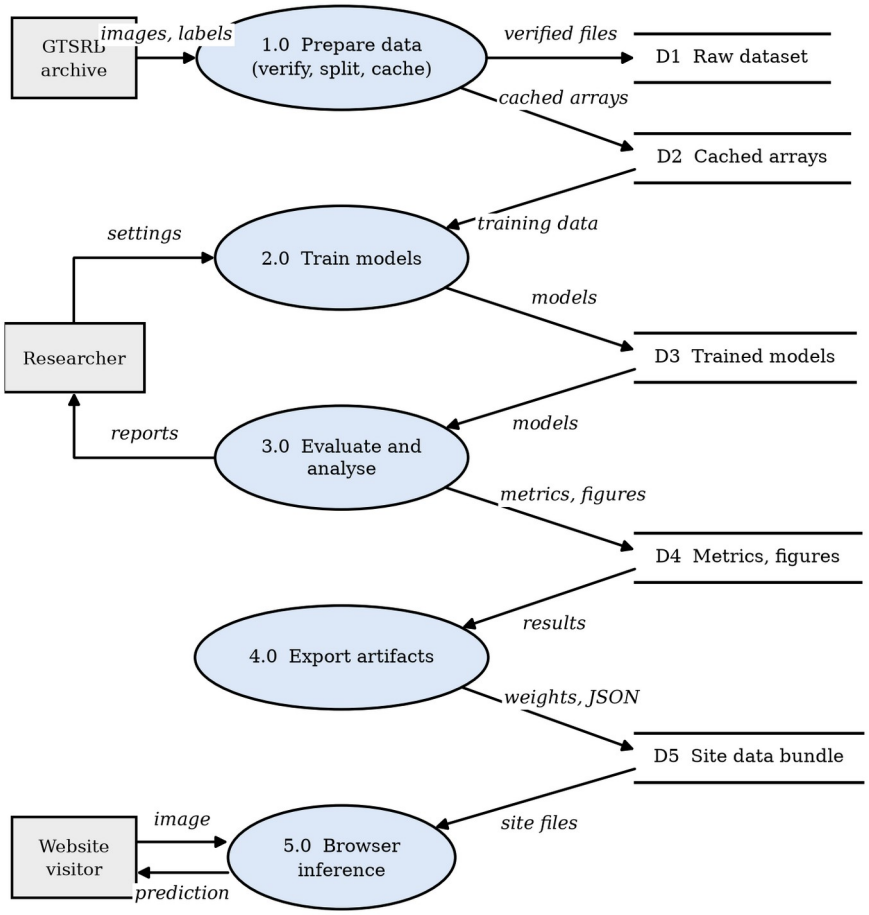


Figure 4.3: Data flow diagram, level 1

CHAPTER 5: RESEARCH METHODOLOGY

5.1 Methodology Used

The study is an experimental one. Figure 5.1 gives the end-to-end workflow, which is executed in a single notebook so that every number in this report can be regenerated.

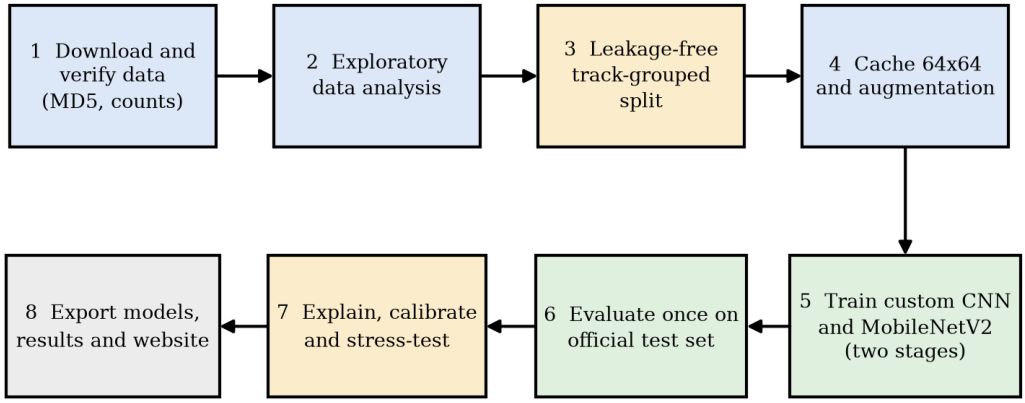


Figure 5.1: End-to-end workflow of the study

5.1.1 Principles

- **Protocol before results.** The split, metrics and selection rule were fixed before models were compared (Section 2.4).
- **One change at a time.** The progress ladder changes a single factor per step, so the effect of each decision is attributable.
- **Validation for decisions, test once.** Early stopping and model selection use validation data. The test set is scored once per model.
- **Assertions over trust.** Data counts, file existence, split properties and reload parity are checked by code that stops the run if violated.
- **Seeds and checksums.** A fixed seed (42) and a checksum-verified archive make runs repeatable on the same hardware.

5.1.2 Evaluation Metrics

Let TP_c, FP_c and FN_c be the true positives, false positives and false negatives of class c among N evaluated images and $C = 43$ classes. The metrics used are:

$$Accuracy = (1 / N) \Sigma_i [\hat{y}_i = y_i] \quad (2)$$

$$P_c = TP_c / (TP_c + FP_c), \quad R_c = TP_c / (TP_c + FN_c), \quad F1_c = 2\,P_c\,R_c / (P_c + R_c) \qquad (3)$$

$$\text{Macro-F1} = (1 / C)\,\Sigma_c\, F1_c \qquad (4)$$

Macro-F1 gives every class the same weight, so rare classes are not hidden behind common ones. Top-3 accuracy counts a prediction as correct if the true class is among the three highest-probability classes. The loss minimised in training is the cross-entropy between the one-hot label and the predicted distribution:

$$L = -\log p(y = y_{true} \mid x) \qquad (5)$$

Calibration is summarised by the expected calibration error [6]. Predictions are grouped into $M = 15$ equal-width confidence bins B_m; with $acc(B_m)$ the accuracy and $conf(B_m)$ the mean confidence in a bin:

$$ECE = \Sigma_m\, (|B_m| / N) \cdot |\, acc(B_m) - conf(B_m)\,| \qquad (6)$$

Selective prediction. For a threshold t the system answers when the confidence is at least t and defers otherwise. Coverage is the fraction of images answered, and selective accuracy is the accuracy on the answered images. Sweeping t traces the trade-off between them [7].

Grad-CAM. For an image and a class c with score y_c, and the activations A^k of the last convolutional layer, the channel weights and the heat-map are [5]:

$$\alpha_k = (1 / Z)\,\Sigma_{i,j}\, \partial y_c / \partial A^k_{ij}, \quad L_{Grad\text{-}CAM} = ReLU(\,\Sigma_k\, \alpha_k\, A^k\,) \qquad (7)$$

5.1.3 Experimental Design

Three families of experiments are run. (i) A **progress ladder** of nine steps measured on validation data: a majority-class baseline, a linear classifier, a multi-layer perceptron, a basic CNN, a CNN with batch normalisation and global pooling, the final CNN with augmentation and dropout, MobileNetV2 with a frozen backbone, MobileNetV2 fine-tuned, and an ensemble of the two best models. (ii) A **final evaluation** of four models on the official test set. (iii) **Reliability studies** of the selected model: per-class analysis, Grad-CAM, calibration, selective prediction and corruption robustness. The training budgets of the three run modes of the notebook are shown in Table 5.1; all reported results use the *full* mode.

Table 5.1: Training budgets of the notebook run modes (epochs are upper limits)

Mode	Training images	Custom CNN	Ladder (linear, MLP)	MobileNetV2 head	MobileNetV2 fine-tune	Purpose
smoke	2,400	2	1	1	1	CPU sanity run
demo	all	12	10	6	8	Quick demonstration
full	all	30	15	12	15	Reported results

5.2 Data Collection and Analysis

5.2.1 Dataset and Acquisition

The study uses the German Traffic Sign Recognition Benchmark [13], obtained as a single 320 MB archive from a public mirror [40]. The notebook downloads the archive, computes its MD5 checksum in streaming fashion, compares it with the published value (7faafd0d78d831c7eb4d555641a83a62) and extracts it only if they match. The archive contains a `Train` folder with one sub-folder per class, a `Test` folder, and CSV files (`Train.csv`, `Test.csv`) that give the original width, height, region of interest and class of every image. File names in the training folders have the form `class_track_frame.png`, which identifies the physical sign (the *track*) that each image belongs to.

Table 5.2: Dataset facts measured by the notebook

Property	Value
Training images	39,209
Official test images	12,630
Classes	43
Physical signs (training tracks)	1,307, with 29 to 30 frames each
Largest class	Speed limit 50 km/h: 2,250 images
Smallest class	Speed limit 20 km/h: 210 images
Class imbalance	10.7 times between largest and smallest
Image width	25 to 243 px; median 43 px; 40 % under 40 px
Aspect ratio (width / height)	Median 1.00; 5th to 95th percentile 0.91 to 1.13
Brightness (grey level, 3,000-image sample)	Mean 82, standard deviation 48; range 7 to 248

5.2.2 Exploratory Data Analysis

Each finding below changed a modelling decision.

Class imbalance. Figure 5.2 shows the number of training images per class. Speed-limit and some warning classes have many images, while several classes have a few

hundred. This is why macro-F1 is reported next to accuracy, and why the relationship between class size and per-class F1 is examined in Chapter 9.

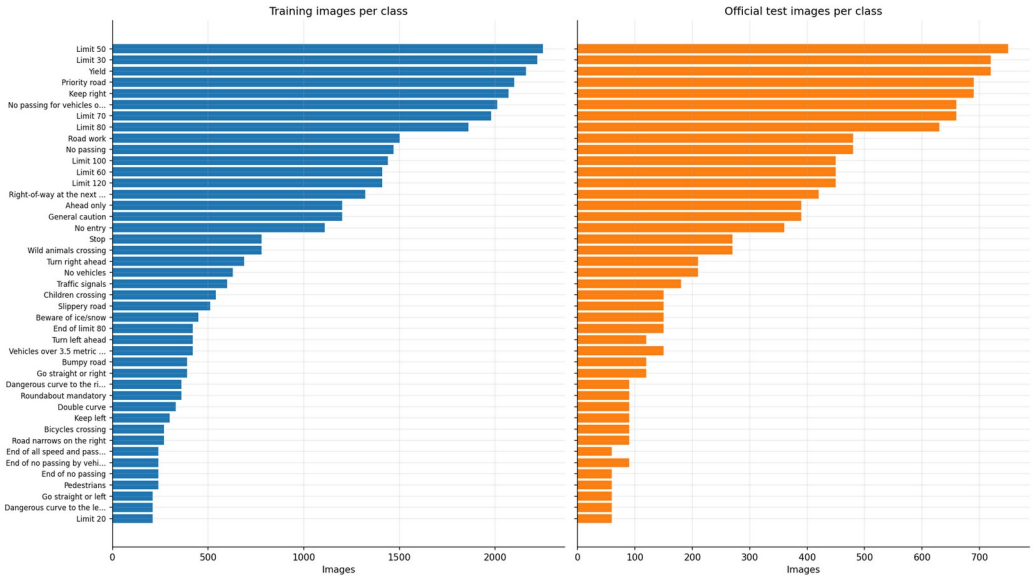


Figure 5.2: Number of images per class in the training and official test sets

Appearance of the classes. Figure 5.3 shows one random training example for each of the 43 classes. Several groups of classes look alike, notably the speed limits, which differ only in the digits.

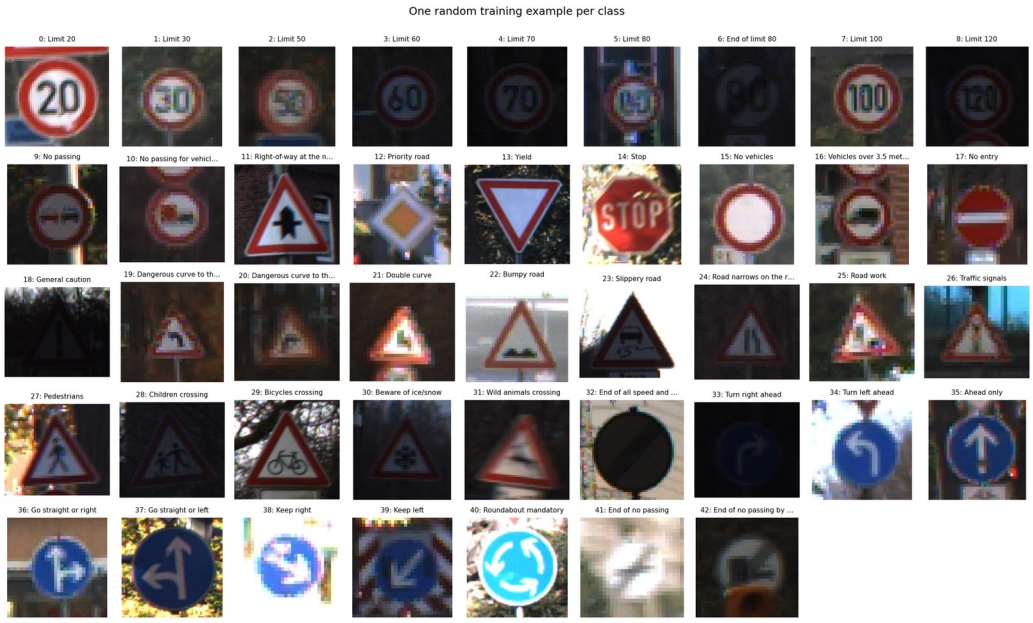


Figure 5.3: One random training example per class

Image size. The images are small (Figure 5.4). The median width is 43 pixels and 40 % are narrower than 40 pixels, so there is little detail to preserve. This supports caching images at 64 x 64 pixels: a higher resolution would only interpolate pixels that do not exist, at higher memory and training cost.

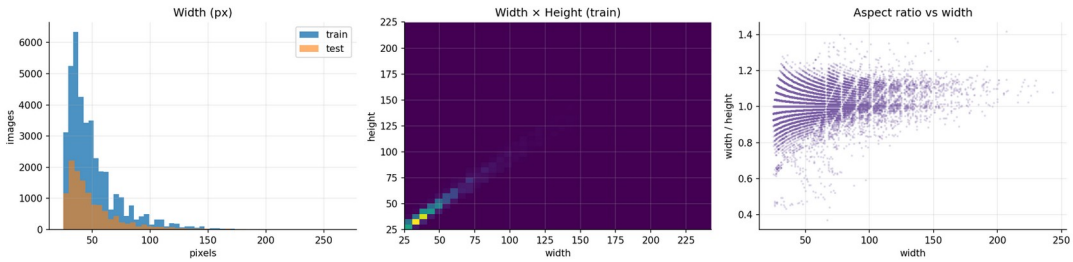


Figure 5.4: Distribution of image width, size and aspect ratio in the training images

Lighting. Brightness varies from almost black to washed out (Figure 5.5). The model must tolerate these changes, which motivates contrast augmentation during training and an explicit darkening and glare test later.

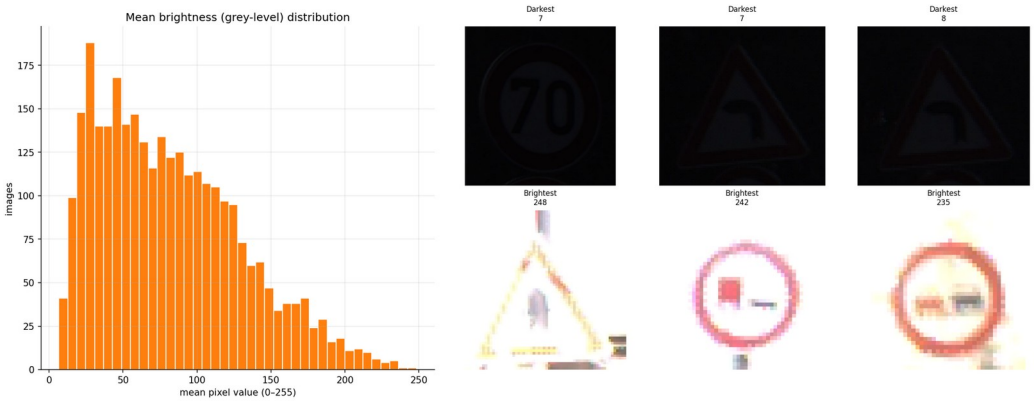


Figure 5.5: Brightness distribution of the training images with the darkest and brightest examples

Track structure. The most important finding is that the images are not independent. The 39,209 training images come from only 1,307 physical signs. Figure 5.6 shows ten frames of a single Stop sign photographed as a vehicle approaches it. Consecutive frames are almost identical in appearance, differing mainly in size. A validation split that separates the frames of one sign would therefore compare the model with images it has effectively already seen.

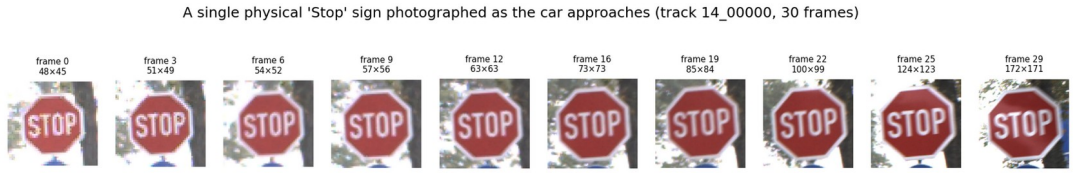


Figure 5.6: Frames of a single physical Stop sign (one training track)

5.2.3 Leakage Analysis

To measure the effect, the notebook compares a random, class-stratified image split with a track-grouped split, each holding out about one seventh of the data. In the random split, 1,291 of the 1,307 tracks have frames on both sides, and every validation track has already been seen in training (100 %). In the grouped split, no track is shared (Figure

5.7). Chapter 7 describes how the grouped split is constructed so that every class also appears in validation.

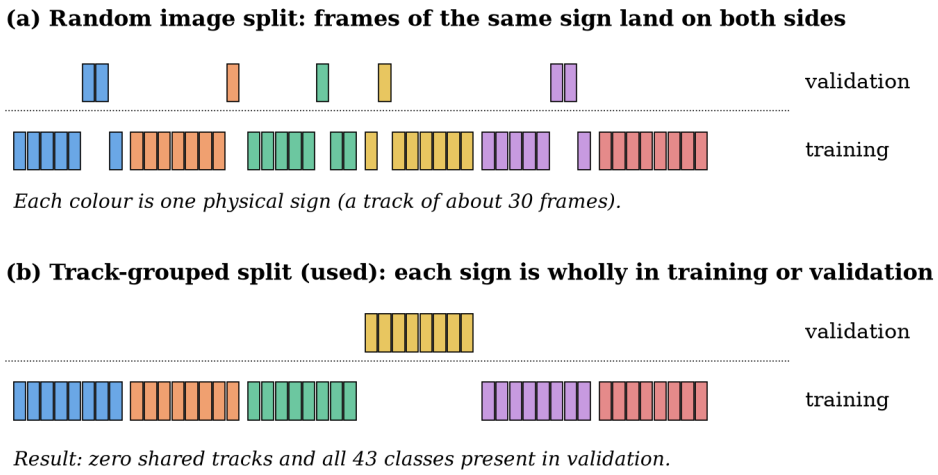


Figure 5.7: Random image split versus the track-grouped split used in this study

5.3 Tools and Technology Used

The tools are listed in Section 4.3. Training and evaluation ran on a Google Colab GPU runtime with TensorFlow 2.21.0 and Keras 3.13.2. The dataset, once cached as 8-bit arrays at 64 x 64 resolution, occupies 0.64 GB, so training streams batches from memory and is limited by the GPU rather than by disk access. Preprocessing and augmentation are layers inside each model, so the exported model accepts raw pixel values and no separate preprocessing code can drift from training.

CHAPTER 6: DESIGN AND ARCHITECTURE

6.1 Structure Chart and Work Breakdown Structure

Figure 6.1 decomposes the project into four work packages: data, modelling, evaluation and delivery. Each package contains the tasks listed under it, and Table 6.1 states the deliverable of each package.

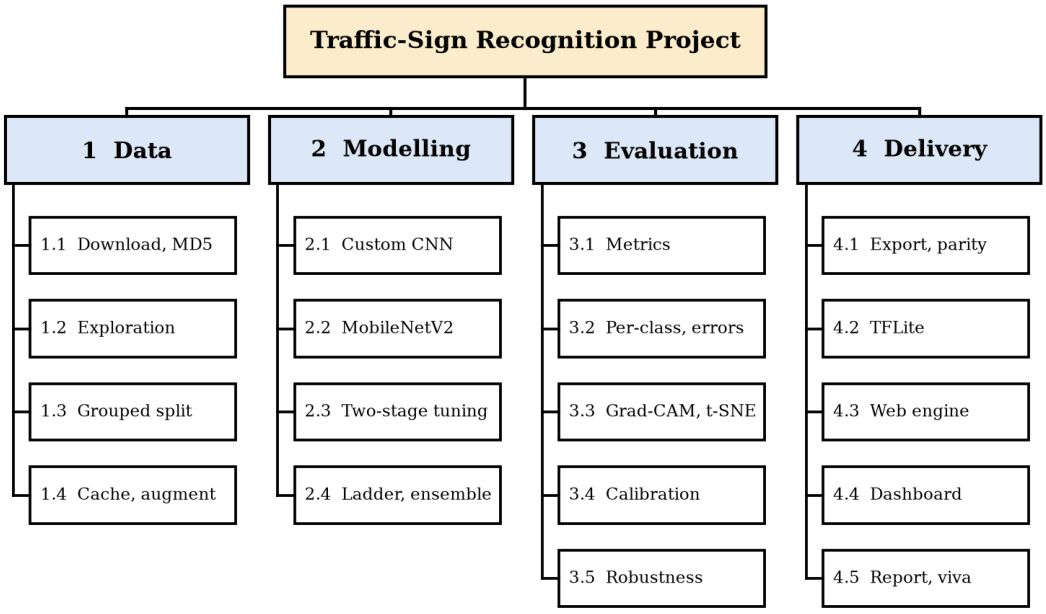


Figure 6.1: Work breakdown structure of the project

Table 6.1: Work packages and deliverables

Work package	Main tasks	Deliverable
1 Data	Download and checksum; exploratory analysis; grouped split; cache and augmentation	Verified, leakage-free training, validation and test arrays
2 Modelling	Custom CNN; MobileNetV2; two-stage tuning; ladder and ensemble	Four trained models and training histories
3 Evaluation	Metrics; per-class analysis and errors; Grad-CAM and t-SNE; calibration; robustness	Result tables and figures
4 Delivery	Export and parity; TFLite; browser engine; dashboard; report and viva material	Results bundle, website, report

6.2 Explanation of Modules

The implementation is divided into the following modules. The Python modules are in the package `traffic_sign_recognition` and are mirrored by self-contained cells of the notebook; the JavaScript modules form the website.

Table 6.2: Modules and their responsibilities

Module	Responsibility
Data acquisition and loading	Download, checksum verification, extraction; build label tables from folder names and CSV files; assert counts and file existence (`data.py`; notebook cells 6 and 7).
Splitting and caching	Hold out whole tracks per class; check zero overlap and full class coverage; resize and cache arrays (`split_train_validation`; notebook cells 19 and 22).
Augmentation	Rotation, translation, zoom and contrast layers active only in training (`models.py`).
Model definitions	Custom CNN and MobileNetV2 builders (`build_custom_cnn`, `build_mobilenet_v2`).
Training	Compile with Adam; early stopping, learning-rate reduction; two-stage fine-tuning (`train.py`; notebook cells 27 to 29).
Evaluation and reporting	Accuracy, macro-F1, top-3, ECE, latency; classification report and confusion matrix (`reporting.py`; cells 31 to 36).
Explainability	Grad-CAM for flat and nested models; embedding extraction and t-SNE (`gradcam.py`; cells 41 and 42).
Reliability	Reliability diagram, coverage curve, corruption functions (cells 44 and 45).
Export	Save models, predictions, tables and figures; reload parity; TFLite conversion (cells 50 to 52).
Browser engine (`nn.js`)	Loads weights; forward pass; Grad-CAM backward pass.
Imaging and charts (`imaging.js`, `charts.js`)	Canvas helpers, corruptions, overlay; SVG and canvas charts with tooltips and table views.
Application (`demo.js`, `dashboard.js`)	Live demo, stress lab, verification; results dashboard.

6.3 Flow Chart and Activity Diagram

Figure 6.2 shows the control flow of the experiment pipeline. The two decision points are the data check at the start, which stops the run if the checksum or counts are wrong, and the early-stopping decision in training, which keeps training while validation accuracy improves and restores the best weights otherwise.

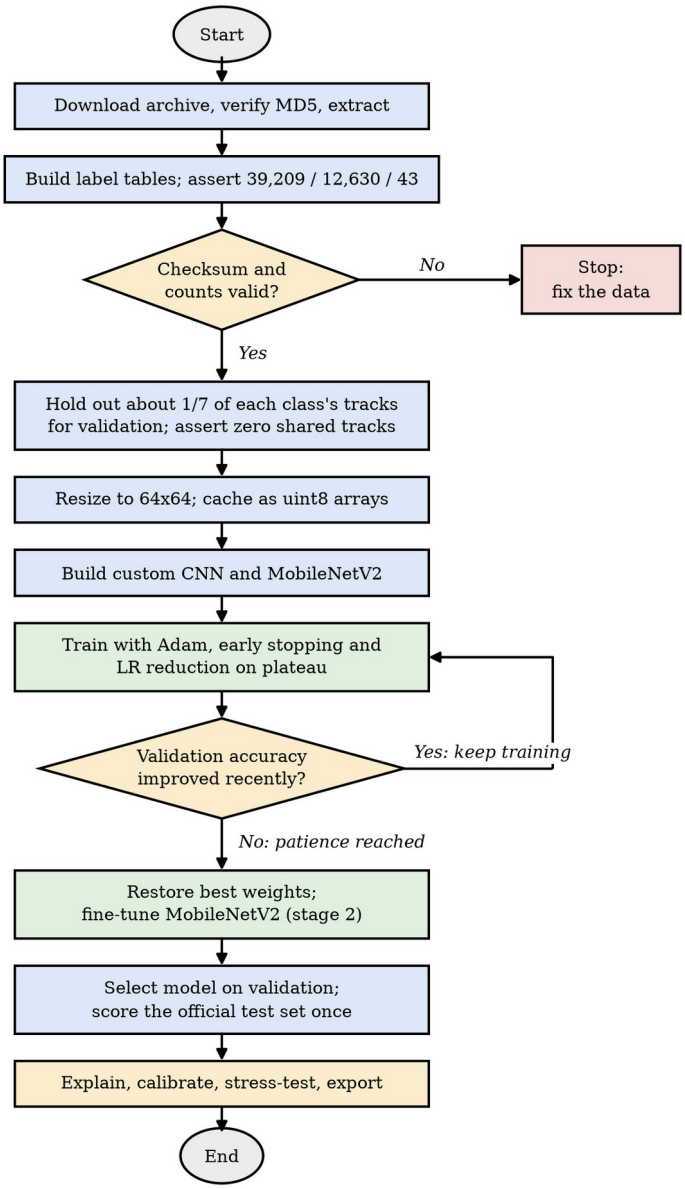


Figure 6.2: Flow chart of the experiment pipeline

The browser application has a second flow, shown in Figure 6.3: files produced by the notebook are converted by a build script into static data files, which the page loads; each image then passes through cropping and optional corruption, the forward pass and Grad-CAM, and the user interface. All of it runs locally.

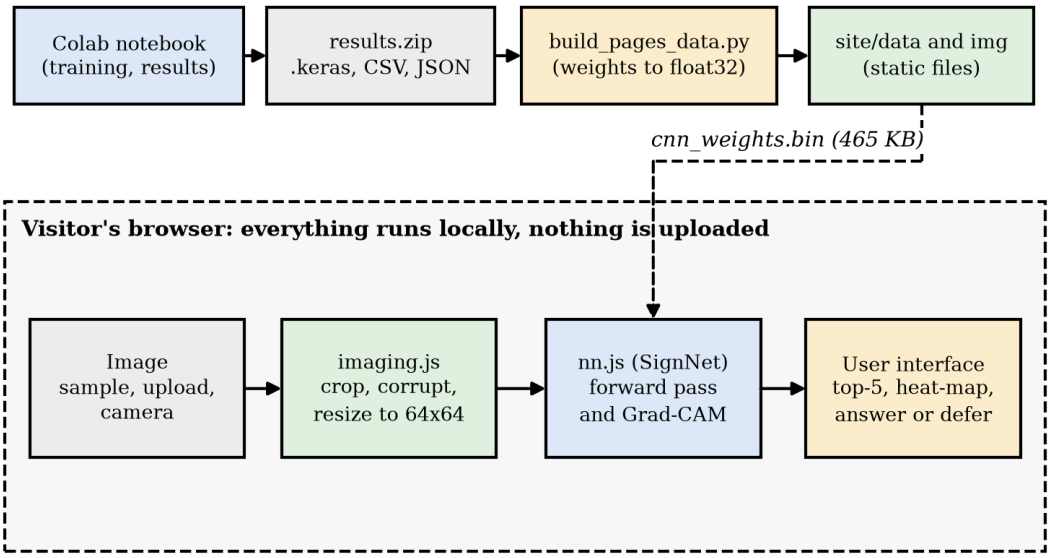


Figure 6.3: Data and control flow of the website

6.4 ER Diagram and Class Diagram

The data handled by the system can be described by the entity-relationship diagram in Figure 6.4. A class contains many tracks, a track contains many images, a model is trained in many runs, each run produces many predictions, and each image appears in many predictions.

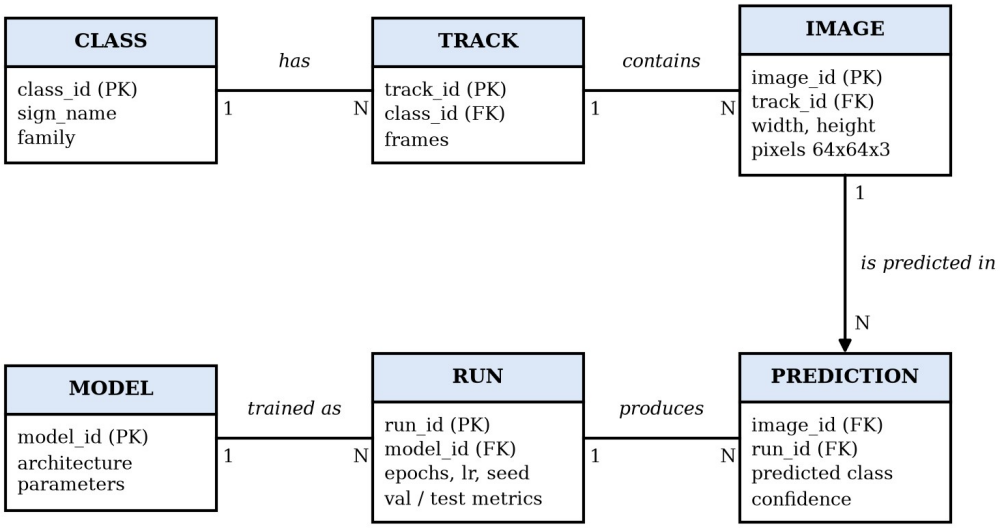
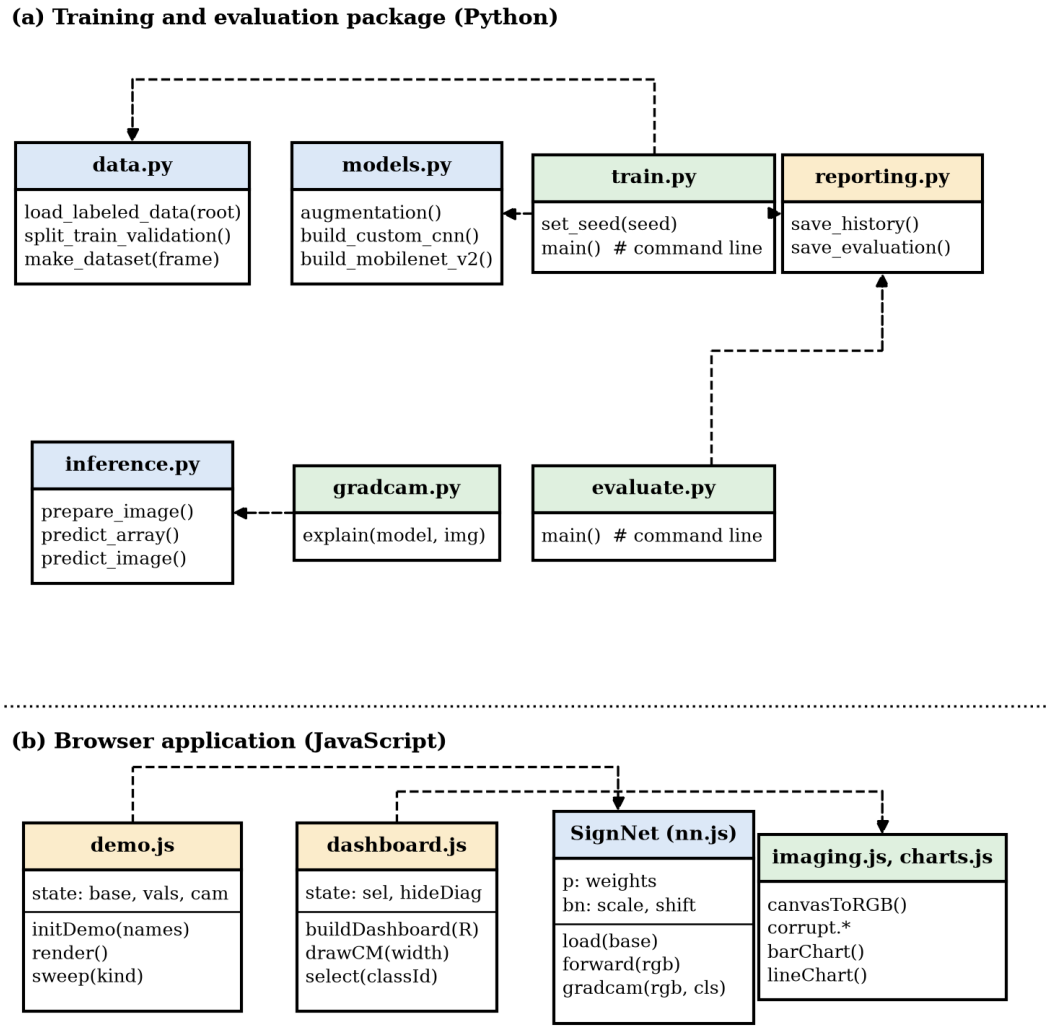


Figure 6.4: Entity-relationship diagram of the data and results

Figure 6.5 shows the structure of the code: the Python package (a) and the browser application (b). The Python package is organised as modules of functions rather than

classes, so its modules are drawn as classes with their principal functions; `SignNet` is the only true class in the browser code.



Dashed arrows denote 'uses' dependencies.

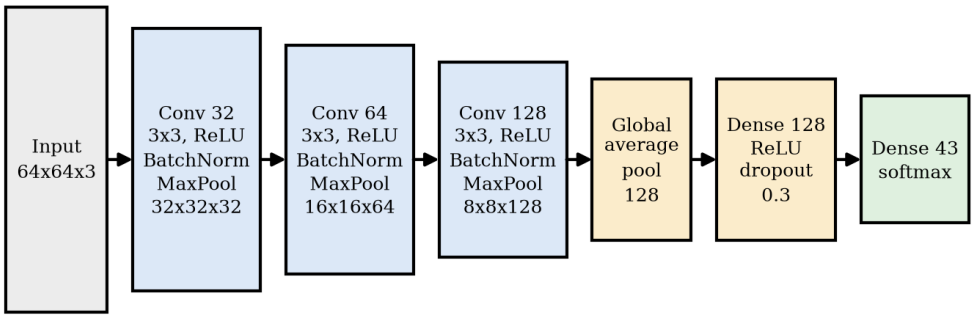
Figure 6.5: Class and module diagram of the code

6.5 Model Architectures and Algorithms

6.5.1 Custom CNN

The custom CNN (Figure 6.6) is intentionally small. It contains three blocks, each a 3 x 3 convolution with ReLU, batch normalisation, 2 x 2 max pooling and dropout (0.2), with 32, 64 and 128 filters. Global average pooling reduces the final 8 x 8 x 128 map to a 128-vector, which passes through a 128-unit dense layer with dropout (0.3) to a 43-way softmax. Augmentation is applied to raw pixels before they are scaled to the range 0 to 1. Table 6.3 lists the layers and parameters; the total is 116,203 parameters, of

which 115,755 are trainable, and about 41 million multiply-accumulate operations per image.



Augmentation is active in training only (rotation, translation, zoom, contrast, before rescaling to [0, 1]).

Figure 6.6: Architecture of the custom CNN

Table 6.3: Layers and parameters of the custom CNN

Layer	Output shape	Parameters
Input (raw pixels, augmentation, rescale)	64 x 64 x 3	0
Conv 3x3, 32 filters, ReLU	64 x 64 x 32	896
BatchNorm, MaxPool 2x2, Dropout 0.2	32 x 32 x 32	128
Conv 3x3, 64 filters, ReLU	32 x 32 x 64	18,496
BatchNorm, MaxPool 2x2, Dropout 0.2	16 x 16 x 64	256
Conv 3x3, 128 filters, ReLU	16 x 16 x 128	73,856
BatchNorm, MaxPool 2x2, Dropout 0.2	8 x 8 x 128	512
Global average pooling	128	0
Dense 128, ReLU; Dropout 0.3	128	16,512
Dense 43, softmax	43	5,547
Total		**116,203**

6.5.2 MobileNetV2 Transfer Learning

The transfer-learning model (Figure 6.7) upsamples the 64 x 64 input to 128 x 128 inside the model, scales it to the range -1 to 1 as MobileNetV2 expects, passes it through the ImageNet-pretrained backbone with average pooling (a 1,280-dimensional vector), applies dropout (0.25) and a 43-way dense layer. It is trained in two stages. In stage 1 the backbone is frozen and only the new dense layer is trained (55,083 parameters). In stage 2 the last 50 backbone layers are unfrozen while all batch-normalisation layers

stay frozen, and training continues at a ten times lower learning rate (1,834,219 of 2,313,067 parameters trainable).

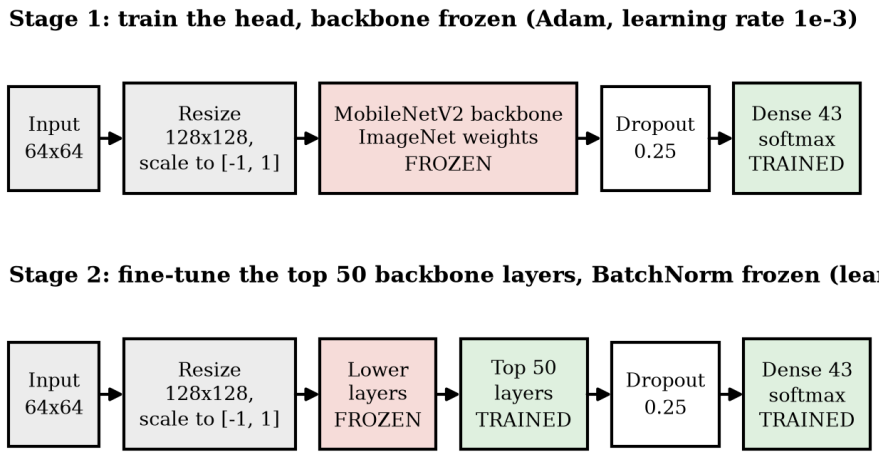


Figure 6.7: Two-stage training of the MobileNetV2 model

6.5.3 Algorithms

Algorithm 6.1 describes the construction of the leakage-free validation split. Algorithm 6.2 describes training with early stopping, and Algorithm 6.3 the Grad-CAM computation used in the browser. Algorithm 6.4 describes selective prediction.

Algorithm 6.1: Per-class, track-grouped validation split

```
Input: table of training images with (label, track); seed s; fraction f = 1/7
Output: fitting set F and validation set V
rng <- random generator seeded with s
V_tracks <- empty list
for each class c in 0..42:
    T <- distinct tracks of class c
    k <- max(1, round(f * |T|))
    V_tracks <- V_tracks + k tracks drawn from T without replacement using rng
V <- images whose track is in V_tracks;   F <- all other images
assert no track occurs in both F and V
assert every class occurs in V
```

Algorithm 6.2: Training with early stopping and learning-rate reduction

```
Input: model, fitting set F, validation set V, learning rate a, patience p, epoch limit E
best <- -infinity;  wait <- 0
for epoch = 1..E:
    update the weights on augmented mini-batches of F with Adam(a)
    v <- accuracy of the model on V (augmentation off)
    if validation loss did not improve for p/2 epochs: a <- 0.4 * a
    if v > best: best <- v; save the weights; wait <- 0
    else: wait <- wait + 1
    if wait >= p: break
restore the saved weights
```

Algorithm 6.3: Grad-CAM for the last convolutional layer (class score before softmax)

```
Input: image x, network with last convolution output A (16 x 16 x 128), target class c
forward pass: obtain A, the logits z and the probabilities p
y_c <- z[c]                          # class score before the softmax
back-propagate dy_c / dA through dense layers, global average pooling,
    max pooling (to the arg-max positions) and batch normalisation
alpha_k <- mean over positions of dy_c / dA[:, :, k]
cam <- ReLU( sum_k alpha_k * A[:, :, k] )
```

```
cam <- cam / (max(cam) + 1e-8)
return cam, upsampled bilinearly to the image size
```

Algorithm 6.4: Selective prediction with an answer threshold

```
Input: probability vector p for an image, threshold t
conf <- max(p);  class <- argmax(p)
if conf >= t: answer class with confidence conf
else: defer to a human or another sensor
Offline: for t in a grid, coverage(t) = mean(conf >= t); accuracy(t) = accuracy on {conf
    >= t}
```

CHAPTER 7: IMPLEMENTATION

This chapter describes how the design of Chapter 6 was implemented: the data pipeline, the models and their training, the evaluation and explanation code, the export step, and the browser implementation. It follows the structure of the research format (data presentation, implementation, analysis) and then gives the screenshots and the source code of selected modules required of development projects.

7.1 Data Presentation and Preparation

7.1.1 Loading and Verification

The notebook builds two label tables. For training, it walks the class folders in sorted order and records, for each image, the path, the class, the track (class and track number taken from the file name) and the frame number; width and height are joined from `Train.csv`. For testing, it reads `Test.csv`. The code then asserts that there are 39,209 training images, 12,630 test images and 43 classes, that every file exists, and, for the archive, that the checksum matches. A failed assertion stops the run before any training, so that a wrong or truncated dataset cannot silently produce results.

7.1.2 The Validation Split

The split is the most consequential implementation detail of the project, and it required two corrections. The first version used the scikit-learn `GroupShuffleSplit` [29], which keeps all frames of a track together but does not balance classes. In the first full run the printed check *all 43 classes in validation* was *False*: five classes (0, 19, 21, 23 and 29) had no validation images, which biased validation accuracy, the ladder and the model selection (the test results were unaffected, because those classes were still trained on and the test set is separate). A first repair with `StratifiedGroupKFold` passed in a local check but failed the same assertion on Colab, because that splitter only attempts to balance classes and its behaviour depends on the library version. The final implementation therefore selects tracks explicitly (Algorithm 6.1): for every class, about one seventh of its tracks (at least one) are chosen with a seeded NumPy generator. It is deterministic, independent of library versions, and guarantees coverage; Listing 7.1 shows the code.

Table 7.1: Random split versus the track-grouped split used

Property	Random image split	Track-grouped split (used)
Validation images	5,602	5,639
Fitting images / tracks	-	33,570 / 1,119
Validation tracks	-	188
Tracks present on both sides	1,291 of 1,307	0
Validation tracks already seen in training	100 %	0 %
Classes present in validation	-	43 of 43 (asserted)

7.1.3 Caching and Augmentation

All images are resized to 64 x 64 pixels with bilinear resampling and stored as 8-bit arrays: 33,570 fitting, 5,639 validation and 12,630 test images, 0.64 GB in total, built in 25 seconds with 16 reader threads. The region of interest given in the CSV is deliberately *not* used to crop: a real detector returns boxes with some margin, so training on the full frame matches deployment better.

Augmentation is implemented as Keras layers placed at the start of each model, so it is active only while training. Table 7.2 lists the settings, and Figure 7.1 shows one image and fifteen augmented versions. Horizontal flipping is deliberately absent: flipping a “turn left ahead” sign yields an image that looks like “turn right ahead” but would keep the original label.

Table 7.2: Augmentation settings (training only)

Transform	Setting	Reason
Rotation	factor 0.04 of a full turn (about 14 degrees either way)	Camera tilt, slanted poles
Translation	5 % of width and height	Imperfect crops
Zoom	8 %	Different distances
Contrast	10 %	Lighting variation
Horizontal flip	Not used	Would change the meaning of directional signs

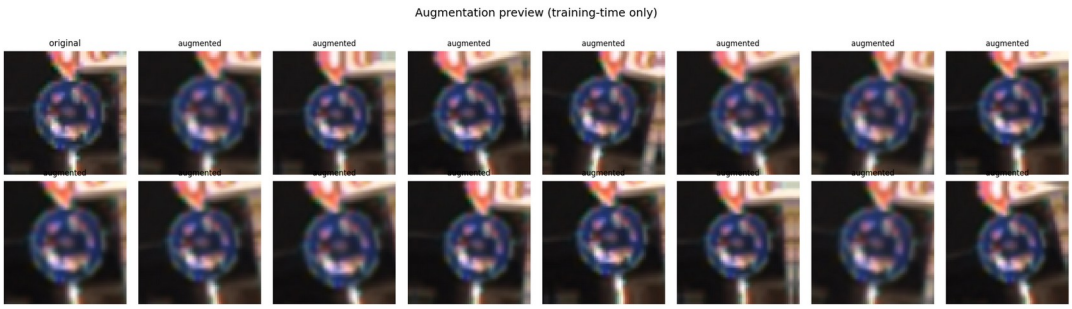


Figure 7.1: Original image and random augmentations (training only)

7.1.4 Hyper-parameters

Table 7.3: Hyper-parameters and training settings of the reported run

Setting	Custom CNN	MobileNetV2 stage 1	MobileNetV2 stage 2
Optimiser	Adam	Adam	Adam
Learning rate	1e-3	1e-3	1e-4
Batch size	128	128	128
Epoch limit	30	12	15
Early stopping (patience, on validation accuracy)	5	4	4
Learning-rate reduction	x0.4 after 2 stalled epochs	x0.4 after 2	x0.4 after 2
Trainable parameters	115,755	55,083	1,834,219
Input to the network	64 x 64	64 x 64, upsampled to 128 x 128	same
Random seed	42	42	42

7.2 Implementation of the Work

7.2.1 Models and Training

Both models are built with the Keras functional API (Listing 7.2). A helper function compiles a model with Adam and sparse categorical cross-entropy, attaches early stopping (monitoring validation accuracy and restoring the best weights) and learning-rate reduction on plateau, trains on the in-memory arrays and returns the history and the elapsed time. For MobileNetV2 the second stage sets the backbone trainable, re-freezes all but its last 50 layers and every batch-normalisation layer, and recompiles with the lower learning rate. If the pretrained weights cannot be downloaded the code falls back to random initialisation with a visible warning, so that the notebook still runs offline.

7.2.2 The Progress Ladder

The ladder trains four additional models of increasing capability on the same fitting and validation sets and with the same optimiser, and reuses the final custom CNN and the two MobileNetV2 models. Table 7.4 defines each step.

Table 7.4: Definition of the progress-ladder steps

Step	Model	What changes
0	Majority-class baseline	Always predict the most frequent training class
1	Linear classifier	One dense softmax layer on 16 x 16 pixels
2	Multi-layer perceptron	Two hidden layers (256, 128 units) on 32 x 32 pixels
3	Basic CNN	Two convolution layers, flatten, dense layer; no normalisation, augmentation or dropout
4	Deeper CNN	Three blocks, batch normalisation, global average pooling
5	Final custom CNN	Adds augmentation and dropout
6	MobileNetV2, frozen	Pretrained backbone, new head only
7	MobileNetV2, fine-tuned	Top 50 backbone layers adapted at a lower rate
8	Ensemble	Average of the probabilities of steps 5 and 7

7.2.3 Evaluation, Explanation and Reliability Code

A scoring function converts a probability matrix into accuracy, macro-F1 (scikit-learn), top-3 accuracy and ECE with 15 bins, so the same code scores the validation and the test set and the ensemble. Latency is the mean time of 200 single-image calls. The per-class report, confusion matrix, most frequent confusions and confident errors are computed from the test predictions. For Grad-CAM the code builds a helper model that returns the last convolutional feature map and the network output; because the MobileNetV2 model contains its backbone as a nested model, the forward pass is carried out layer by layer around it. Embeddings for t-SNE are taken from the input of the last dense layer. The corruption functions apply brightness scaling, Gaussian noise, Gaussian blur, rotation with border replication and a black occluding square to 6,000 test images at three severities each.

7.2.4 Export and Parity Checks

The export step saves each model in the native Keras format together with its metrics, classification report, confusion matrix, training history and per-image predictions; the comparison, ladder, per-class, confusion and robustness tables; a 1,000-image sample of the test set; and a README that shows how to load a model. The two main models are then reloaded and their predictions on 256 images are compared with those of the

in-memory models; a difference above 10^{-4} stops the run. The selected model is also converted to TensorFlow Lite with 16-bit float weights and evaluated on 500 test images.

7.2.5 The Browser Implementation

To let anyone run the selected model without installation or a server, the trained network was re-implemented in plain JavaScript (`nn.js`, 150 lines, no libraries). A build script (`build_pages_data.py`) reads the weights directly from the saved Keras file and writes them as a single 465 KB float32 file with a JSON manifest that records each array's name and shape and the batch-normalisation epsilon. In the browser the weights are folded into per-channel scale and shift constants for the normalisation layers. The forward pass is a direct implementation of the architecture: a 3 x 3 convolution with ReLU, batch normalisation, 2 x 2 max pooling (recording the winning positions), three times, then global average pooling and two dense layers (Listing 7.3).

Grad-CAM is implemented by hand-written back-propagation from the chosen class score to the output of the last convolutional layer, through the dense layers, the ReLU gate, global average pooling, the recorded max-pooling positions and the batch-normalisation scale (Listing 7.4). The channel weights are the spatial means of these gradients; the heat-map is the ReLU of the weighted sum of the activations, scaled to the range 0 to 1 and upsampled bilinearly. The same code supports two scores: the class logit (default) and the class probability (as in the notebook), which allowed the numerical comparison in Section 9.2.4.

The page itself is written as ES modules: `imaging.js` (crop, corruptions, overlay), `charts.js` (an SVG and canvas chart kit with tooltips on hover and keyboard focus and a table view for every chart), `demo.js` (the demonstration and stress lab) and `dashboard.js` (the result charts). A second build step turns the notebook's results bundle into `results.json` and a sprite of 116 sample test images (86 correctly classified and 30 misclassified by the Keras model, with the Keras confidence recorded for each). A GitHub Actions workflow publishes the `site` folder to GitHub Pages on every change.

7.3 Analysis and Interpretation of Training

Table 7.5 records the training cost of each stage and Figure 7.2 and Figure 7.3 show the learning curves.

Table 7.5: Training cost of the reported run

Stage	Epochs run (limit)	Time	Best validation accuracy
Custom CNN	29 (30)	341 s	97.61 %
MobileNetV2 stage 1 (head)	12 (12)	244 s	72.58 %
MobileNetV2 stage 2 (fine-tune)	14 (15)	422 s	90.55 %
Ladder: linear classifier	15 (15)	17 s	70.19 %
Ladder: MLP	15 (15)	20 s	83.24 %
Ladder: basic CNN	25 (30)	107 s	93.37 %
Ladder: deeper CNN	24 (30)	150 s	95.14 %

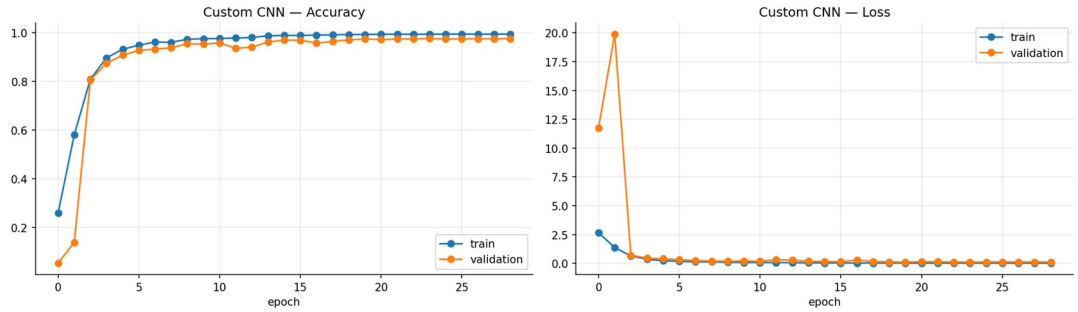


Figure 7.2: Training and validation curves of the custom CNN

The custom CNN's validation accuracy tracks its training accuracy closely: at the last epoch the training accuracy is 99.45 % and the validation accuracy 97.59 %, a gap of under two points. Training accuracy is lower than validation accuracy in the first epochs because augmentation and dropout are active only during training. Early stopping ended training after 29 of at most 30 epochs.

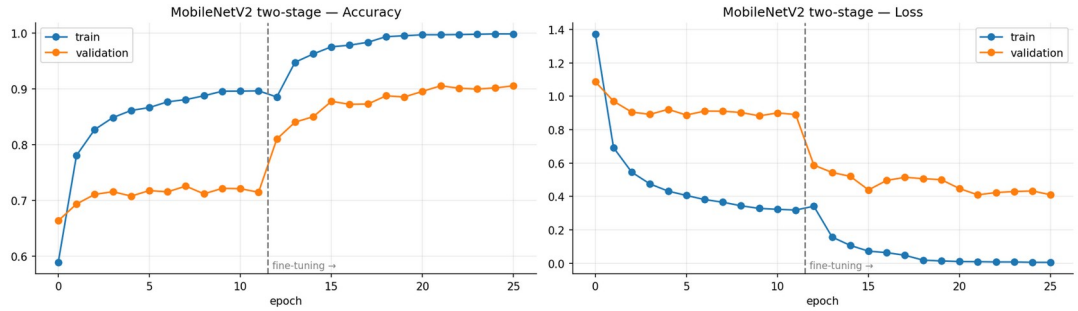


Figure 7.3: Training and validation curves of MobileNetV2 across the two stages (the dashed line marks the start of fine-tuning)

For MobileNetV2 the head-only stage plateaus: its validation accuracy stays near 72 % although the training accuracy reaches 89.6 %. Fine-tuning raises validation accuracy sharply to 90.5 %, but the training accuracy reaches 99.8 %: the fine-tuned model over-fits, memorising the training images far more than the small CNN does. This agrees with the expectation that the pretrained features are not matched to tiny sign crops and

that a network with 2.3 million parameters has more capacity than 33,570 highly correlated images can constrain.

7.4 Screenshots

The website is published at https://briskam.github.io/mait-mini/. Figure 7.4 to Figure 7.8 show the main views. In Figure 7.4 the hero panel cycles real test images through the model; each frame shows the image, the Grad-CAM heat-map, the prediction, the confidence and whether it is correct. Figure 7.5 shows a misclassified test image: the true class is Speed limit 120 km/h, the model answers Speed limit 100 km/h with 91.6 % confidence and the page states that the Keras model from the notebook makes the same mistake.

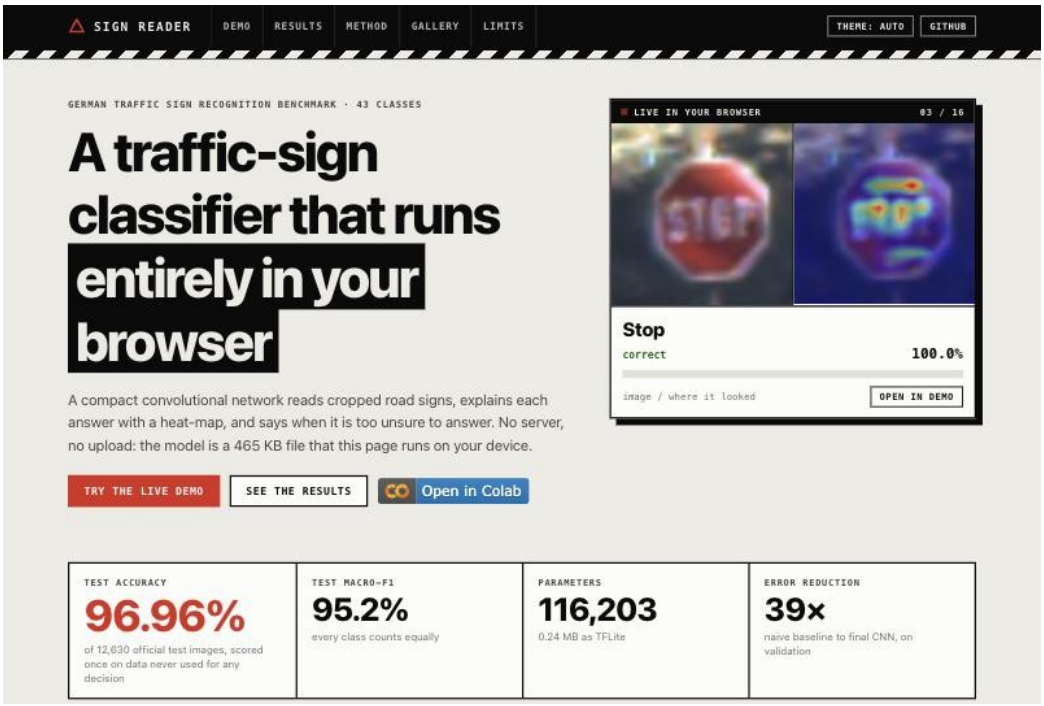


Figure 7.4: Website home view with the live prediction panel and headline results

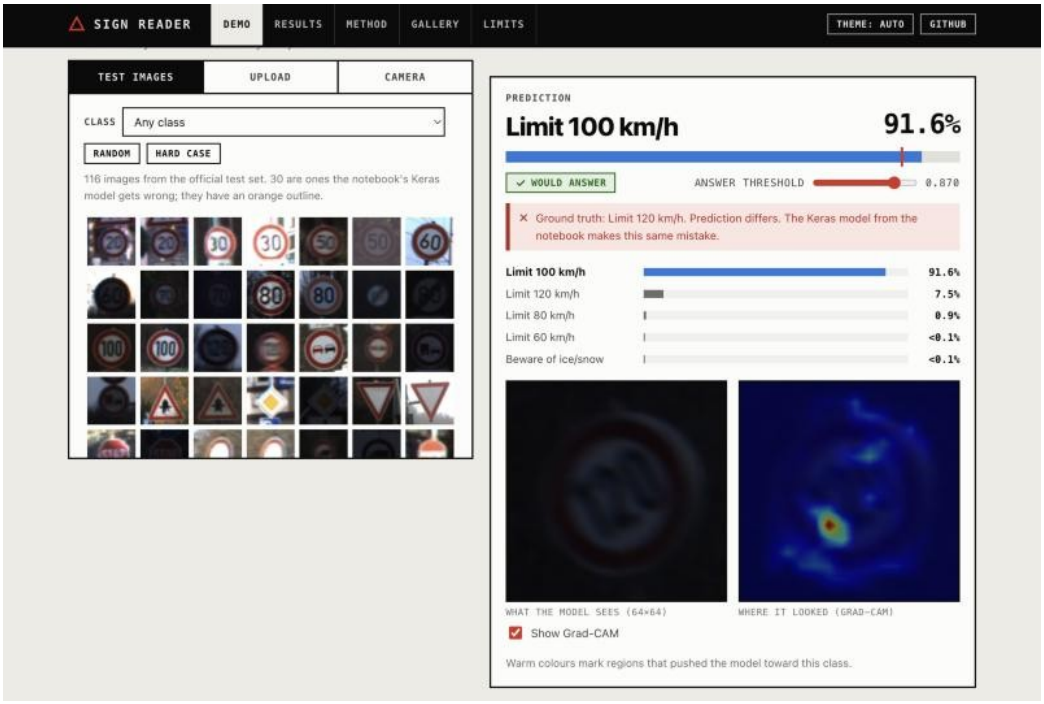


Figure 7.5: Live demonstration of a misclassified test image with Grad-CAM

Figure 7.6 shows the effect of added noise on another test image: the prediction changes to the wrong class with 80 % confidence, which is below the answer threshold of 0.87, so the page reports that the system would defer to a human. Figure 7.7 shows the stress lab with a sweep of the noise level, which plots the model's probability for its clean prediction (and for the true class when they differ) across the range.

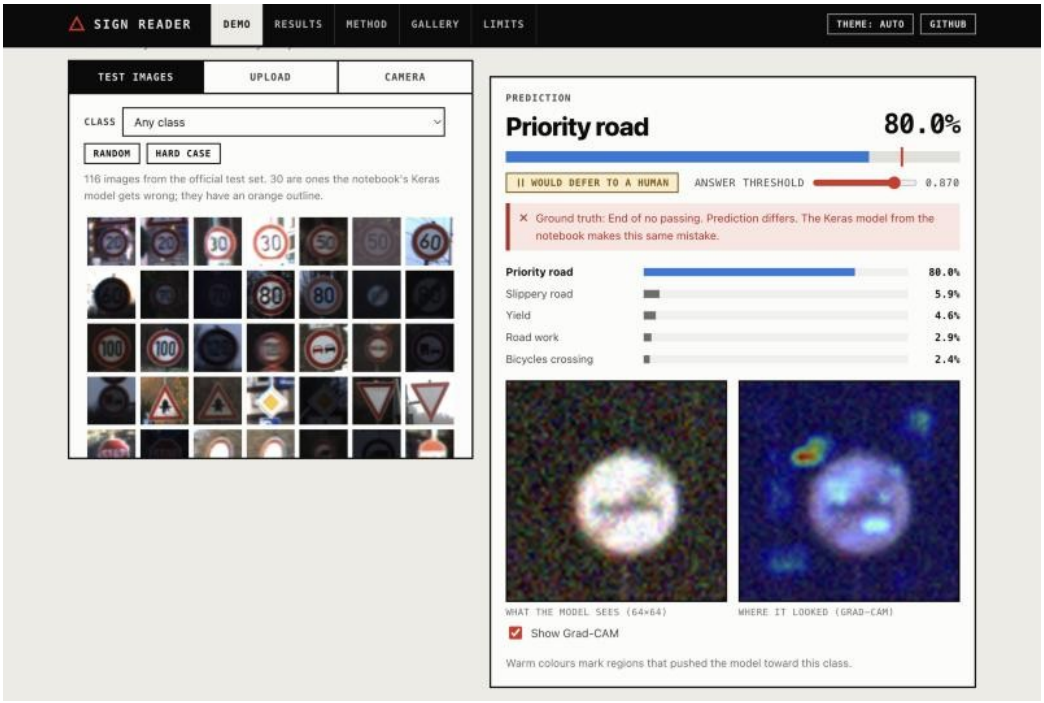


Figure 7.6: Noise applied to a test image; the model would defer

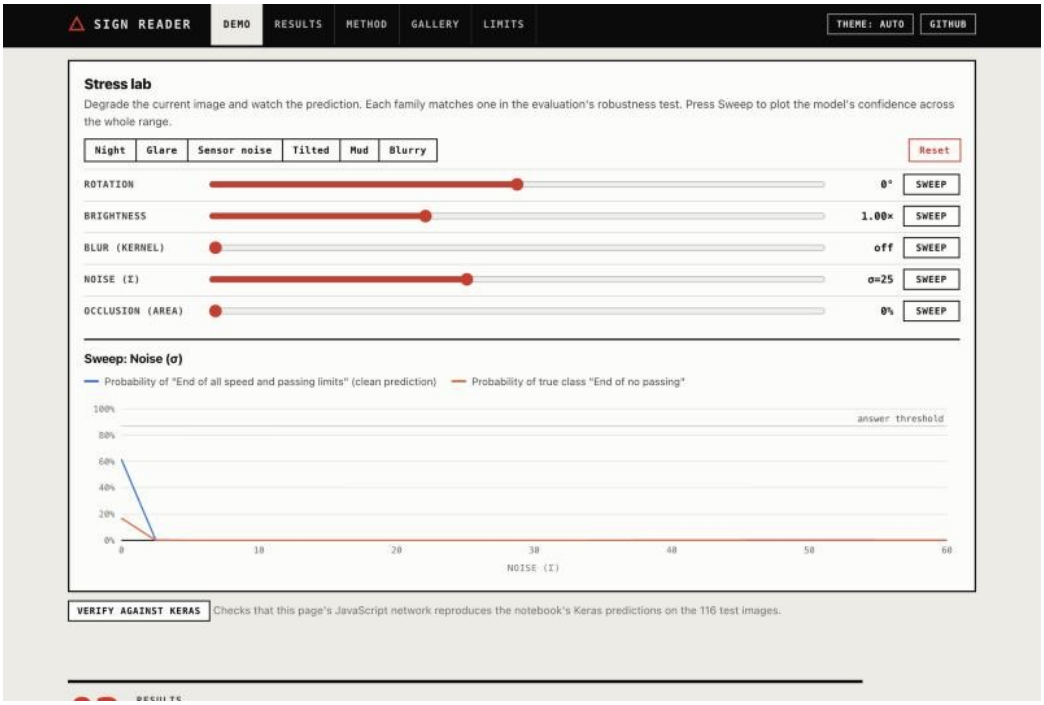


Figure 7.7: Stress lab with presets, sliders and a noise sweep

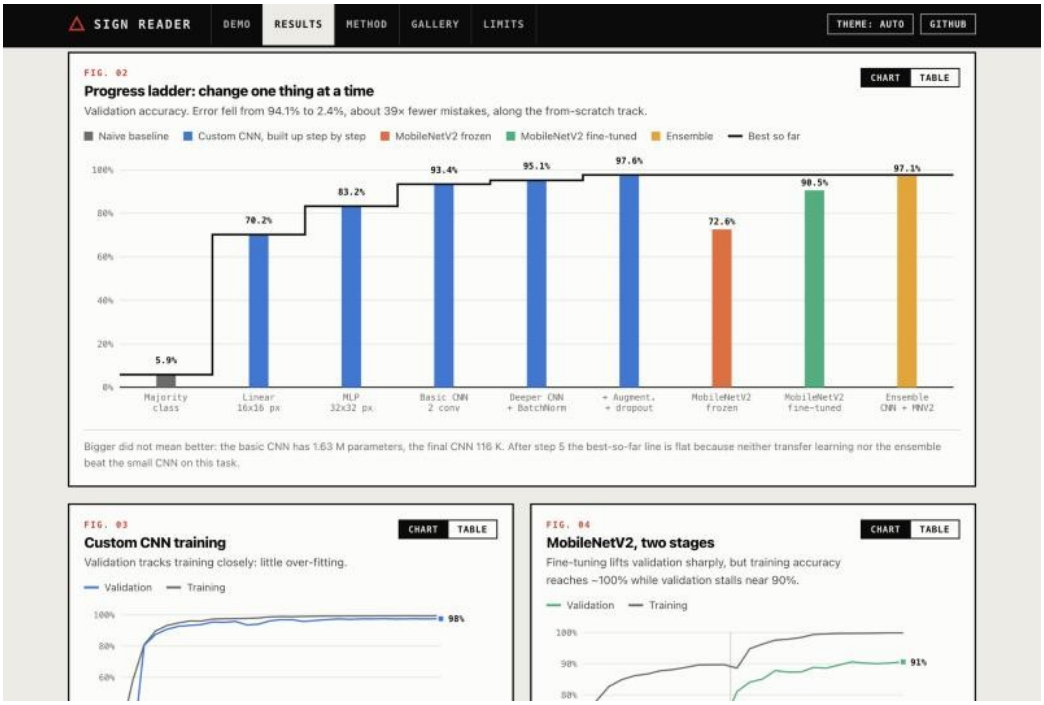


Figure 7.8: Results dashboard: the progress ladder and training curves

7.5 Source Code of Some Modules

The listings below are the most important parts of the implementation. Further code is in the repository and in Appendix A.

Listing 7.1: Per-class, track-grouped validation split (notebook)

```
rng = np.random.default_rng(SEED); val_groups = []
for _, tracks in train_df.groupby("label").group.unique().items():
    # hold out ~1/7 of each class's tracks (at least one)
    val_groups += list(rng.choice(tracks, max(1, round(len(tracks) / 7)), replace=False))
```

```
is_val = train_df.group.isin(val_groups).values
fit_df, val_df = train_df[~is_val].reset_index(drop=True),
    train_df[is_val].reset_index(drop=True)
assert not set(fit_df.group) & set(val_df.group), "Leakage: a track appears in both
    splits"
assert val_df.label.nunique() == NUM_CLASSES, "Validation is missing some classes"
```

Listing 7.2: Custom CNN definition (Keras functional API)

```
def build_custom_cnn():
    inp = layers.Input((IMG, IMG, 3), name="image")           # raw pixels 0-255
    x = augmentation()(inp)                                   # active in training only
    x = layers.Rescaling(1 / 255.0)(x)
    for i, f in enumerate((32, 64, 128), 1):
        x = layers.Conv2D(f, 3, padding="same", activation="relu", name=f"conv{i}")(x)
        x = layers.BatchNormalization()(x); x = layers.MaxPooling2D()(x); x =
            layers.Dropout(0.2)(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation="relu", name="embedding")(x); x =
        layers.Dropout(0.3)(x)
    return keras.Model(inp, layers.Dense(NUM_CLASSES, activation="softmax",
        name="probs")(x), name="custom_cnn")
```

Listing 7.3: Forward pass of the network in JavaScript (nn.js, abridged)

```
forward(rgb, keep = false) {
  let x = Float32Array.from(rgb, (v) => v / 255), H = 64, Cin = 3; const stages = [];
  for (const [i, Cout] of [[1, 32], [2, 64], [3, 128]]) {
    const a = conv3x3(x, H, H, Cin, p[`conv${i}.w`], p[`conv${i}.b`], Cout);   // conv +
        ReLU
    const bn = batchNorm(a, Cout, this.bn[i].scale, this.bn[i].shift);
    const pooled = maxPool(bn, H, H, Cout);                  // keeps arg-max positions
    stages.push({ a, bn, pooled, H, C: Cout }); x = pooled.out; H >>= 1; Cin = Cout;
  }
  const g = globalAveragePool(x, H * H, Cin);                   // 128 numbers
  const h1 = relu(dense(g, p['dense1.w'], p['dense1.b']));     // 128 units
  const probs = softmax(dense(h1, p['dense2.w'], p['dense2.b']));   // 43 classes
  return keep ? { probs, stages, h1pre, n } : { probs };
}
```

Listing 7.4: Grad-CAM back-propagation in JavaScript (nn.js, abridged)

```
const dz = new Float32Array(43);                 // gradient of the score w.r.t. the
    logits
for (let j = 0; j < 43; j++) dz[j] = score === "prob" ? probs[c] * ((j === c) - probs[j])
    : (j === c ? 1 : 0);
dh1pre[i] = h1pre[i] > 0 ? sum_j W2[i][j] * dz[j] : 0;      // dense 2 and ReLU gate
dg[i]     = sum_o W1[i][o] * dh1pre[o];                    // dense 1
for (o of pooledPositions) dB[arg[o]] += dg[o % C] / n;     // pooling and global average
alpha[c] = (sum over positions of dB[.., c]) * bnScale[c] / (H * H);
cam = relu( sum_c alpha[c] * A[.., c] );  cam /= max(cam) + 1e-8;
```

CHAPTER 8: TESTING

The project has no separate unit-test suite for the Python package. Testing was instead built into the pipeline as assertions, into the export step as parity checks, and into dedicated verification scripts and manual test cases for the website. This chapter describes each level and lists test cases with their outcomes.

8.1 Test Strategy

Table 8.1: Levels of testing used

Level	What is tested	How
Data integrity	Archive, counts, files, split properties	Assertions that stop the notebook
Component	Models, scoring, export	Reload parity, TFLite parity, shape and parameter checks
Numerical	Browser network versus Keras	Verification script on 116 images; in-page check
System (functional)	Website behaviour end to end	Manual test cases in a browser (Section 8.4)
Non-functional	Robustness, calibration, layout, accessibility	Corruption study; narrow-viewport and theme checks
Regression	Behaviour after each change	Re-running the notebook; re-running the verification script

8.2 Checks Embedded in the Pipeline

Table 8.2: Automated checks in the notebook and their outcome in the reported run

Check	Expected	Outcome
Archive MD5 equals 7faafd0d78d831c7eb4d555641a83a62	Equal	Passed
Training images, test images, classes	39,209; 12,630; 43	Passed
All image files exist	True	Passed
No track in both fitting and validation sets	Empty intersection	Passed
All 43 classes present in validation	True	Passed (failed in the first run, which led to the corrected split)
Reloaded custom CNN reproduces predictions on 256 images	Difference below 1e-4	Passed (difference 0)
Reloaded fine-tuned MobileNetV2	Difference below 1e-4	Passed (difference 0)
Reloaded ensemble model	Difference below 1e-4	Passed (difference 0)
TFLite model versus Keras on 500 test images	Same predictions	100 % agreement; 97.6 % accuracy on those images

8.3 Numerical Verification of the Browser Network

The correctness of the JavaScript network is critical, because the website presents it as the trained model. It was verified in three ways. First, a Keras model with the identical architecture was built outside the notebook, the weights were loaded from the saved file, and its predictions on the 116 sample images were compared with the confidences that the notebook had recorded for the original model: the largest difference was 1.3×10^{-5}, with identical predictions. This validates both the weight mapping and the batch-normalisation constants. Second, the JavaScript network was run in Node.js on the same images and compared with this Keras reference (Table 8.3). Third, the web page contains a *Verify against Keras* button that repeats the comparison with the values recorded in the notebook, so the check can be repeated by any visitor.

Table 8.3: Agreement between the JavaScript network and Keras on 116 test images

Quantity	Result
Predictions identical	116 of 116
Largest difference in any class probability	3.2 x 10^-6
Largest difference in the Grad-CAM map (class-logit score)	3.1 x 10^-6
Largest difference in the Grad-CAM map (probability score), on the 66 images with confidence below 0.9999	5.9 x 10^-4
Images with confidence of at least 0.9999 (probability-score map not meaningful)	50 of 116
In-page check against notebook confidences	116 of 116 identical; largest difference 1.3 x 10^-5

The check also revealed a property of the notebook's Grad-CAM implementation that is reported in Section 9.2.4: for the 50 near-certain images the probability-based maps differ between two correct implementations because their gradients are of the order of floating-point rounding.

8.4 Functional Test Cases for the Website

Table 8.4 lists the test cases executed on the deployed site (and on the local copy during development), using the browser's console to confirm that no errors were logged.

Table 8.4: Functional test cases for the website

ID	Feature	Steps	Expected result	Result
TC01	Page load	Open the site	Model loads; status reads ready; no console errors	Pass
TC02	Hero panel	Wait on the home view	Test images cycle with prediction and heat-map; counter advances	Pass

ID	Feature	Steps	Expected result	Result
TC03	Select a sample	Click a thumbnail	Prediction, confidence, ground truth and heat-map appear	Pass
TC04	Hard case	Press Hard case	A misclassified image is shown; page notes Keras makes the same mistake	Pass
TC05	Answer threshold	Move the threshold slider	Badge switches between Would answer and Would defer; coverage chart updates	Pass
TC06	Preset	Press Night or Sensor noise	Sliders move; image darkens or noisy; prediction updates	Pass
TC07	Sweep	Press Sweep for noise or brightness	A confidence-versus-severity line chart appears	Pass
TC08	Verify against Keras	Press the verify button	Reports 116 of 116 identical; difference about 1e-5	Pass
TC09	Upload	Provide an image file	Crop box appears; prediction updates; ground truth hidden	Pass (synthetic image)
TC10	Camera	Choose Camera and allow access	Live frame with crop box; periodic prediction	Not tested (no camera in the test environment)
TC11	Class inspection	Click a bar in the F1 chart	Class detail appears with confusions; matrix row highlighted	Pass
TC12	Metric switch	Select a different metric chip	Bar chart redraws for the metric	Pass
TC13	Table view	Press Table on a chart card	Data table replaces the chart	Pass
TC14	Theme	Cycle Theme auto, light, dark	Colours switch; charts remain legible	Pass
TC15	Narrow layout	Set a 375 px viewport	Single column; no horizontal overflow	Pass
TC16	Keyboard	Tab to a chart mark	Tooltip appears on focus	Pass (spot check)
TC17	Deployment	Request the published URL and data files	All files return HTTP 200; page works over HTTPS	Pass
TC18	Hidden state	Switch from a sample to an upload	Ground-truth box disappears	Pass (after fix, Section 8.5)

8.5 Defects Found and Fixed

Table 8.5: Defects found during development and testing

No.	Defect	Detection	Resolution
1	Five classes had no validation images with `GroupShuffleSplit`; validation metrics and model selection were biased	Printed check *all 43 classes in validation: False*; reproduced from file names	Replaced by the explicit per-class track selection and added an assertion
2	A first repair with `StratifiedGroupKFold` passed locally but failed on Colab	Assertion error in the notebook	Deterministic NumPy-based selection independent of library version
3	Grad-CAM from the class probability was numerically unreliable for near-certain predictions	Keras-versus-JavaScript map comparison	Class-logit score used in the browser; difference documented
4	Ladder chart labels overlapped; Grad-CAM titles truncated speed limits	Visual inspection of results	Shorter labels and speed-limit abbreviations
5	A display rule overrode the HTML hidden attribute, leaving a ground-truth box visible after switching to an upload	TC18 observation	Global rule that makes the hidden attribute win

No.	Defect	Detection	Resolution
6	Figure counter also numbered the Chart/Table toggle; headline highlight clipped a letter	Visual inspection	Selector restricted; line height adjusted

8.6 Limitations of the Testing

The Python package has no automated unit-test suite, so its modules rely on the assertions and the end-to-end run. The camera path of the website was not exercised. The website was not tested on Safari, on a range of physical phones or with assistive technology beyond keyboard focus. Timing figures were measured on the development laptop and on a Colab GPU and are not benchmarks of target hardware. Adding a pytest suite for the data and export code and automated browser tests for the website are listed as future work.

CHAPTER 9: RESULTS AND DISCUSSION

All results in this chapter come from one complete run of the notebook in *full* mode on a Google Colab GPU runtime (TensorFlow 2.21.0, Keras 3.13.2, seed 42), unless stated otherwise. The test-set figures were computed once per model; validation figures are labelled as such.

9.1 Results

9.1.1 Model Comparison

Table 9.1 compares the four models on the official test set. The custom CNN, which was chosen on validation accuracy (97.61 % against 90.55 % for fine-tuned MobileNetV2 and 97.07 % for the ensemble), achieves the highest test accuracy among the single models and the best calibration, with 116,203 parameters.

Table 9.1: Performance of the four models on the official test set (12,630 images)

Model	Val. acc.	Test acc.	Macro-F1	Top-3	ECE	Parameters	ms / image
Custom CNN (selected)	97.61 %	**96.96 %**	**95.21 %**	99.03 %	**0.0105**	116,203	18.0
MobileNetV2, frozen	72.58 %	75.16 %	67.46 %	91.18 %	0.0413	2,313,067	153.0
MobileNetV2, fine-tuned	90.55 %	91.26 %	86.59 %	98.11 %	0.0537	2,313,067	153.0
Ensemble (CNN + MobileNetV2)	97.07 %	96.88 %	95.07 %	99.48 %	0.0267	2,429,270	171.0

Fine-tuning improves MobileNetV2 by 16.1 points of test accuracy (75.16 % to 91.26 %) but it still ends 5.7 points below the custom CNN. The ensemble does not beat the best single model: it is slightly lower on both validation (97.07 % against 97.61 %) and test (96.88 % against 96.96 %), and its calibration is worse (ECE 0.0267 against 0.0105); its only advantage is a higher top-3 accuracy (99.48 % against 99.03 %). Latency was timed through single un-compiled calls on a GPU runtime and therefore includes Python overhead; it should be used to compare the models with each other, not as a device benchmark. On this basis MobileNetV2 is about eight to nine times slower than the CNN.

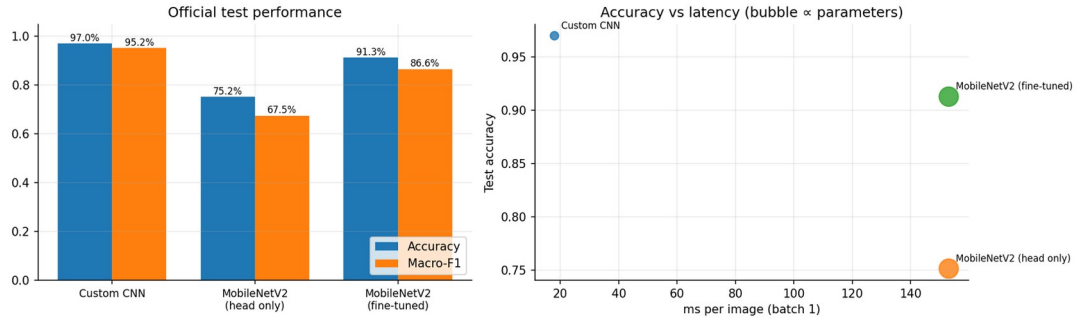


Figure 9.1: Test accuracy and macro-F1 of the models, and accuracy versus latency

9.1.2 Progress Ladder

Table 9.2 and Figure 9.2 show the ladder, measured on validation data. Along the from-scratch track every step improves the result: from 5.85 % for the majority-class baseline to 97.61 % for the final CNN, a reduction of the error rate from 94.1 % to 2.4 %, a factor of about 39.

Table 9.2: Progress ladder: validation accuracy and macro-F1 at each step

Step	Model	Val. accuracy	Val. macro-F1	Parameters
0	Majority-class baseline	5.85 %	0.3 %	0
1	Linear classifier on 16x16 pixels	70.19 %	61.1 %	33,067
2	MLP on 32x32 pixels	83.24 %	78.0 %	825,131
3	Basic CNN (2 conv layers)	93.37 %	91.8 %	1,630,699
4	Deeper CNN + BatchNorm + global pooling	95.14 %	93.8 %	116,203
5	Final CNN (+ augmentation, dropout)	97.61 %	96.9 %	116,203
6	MobileNetV2, frozen backbone	72.58 %	65.3 %	2,313,067
7	MobileNetV2, fine-tuned	90.55 %	88.9 %	2,313,067
8	Ensemble (CNN + MobileNetV2)	97.07 %	96.7 %	2,429,270

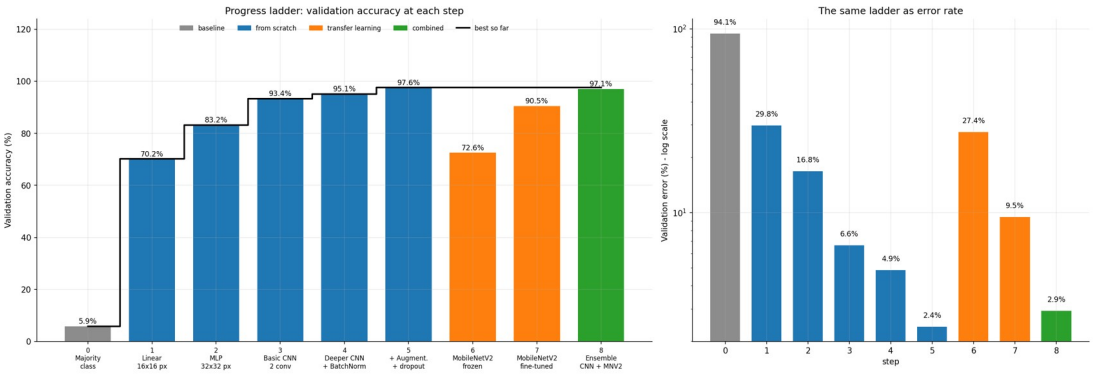


Figure 9.2: Progress ladder: validation accuracy at each step (left) and the same data as error rate on a logarithmic scale (right)

The largest single gains after the linear model come from adding a hidden layer (+13.0 points), convolutions (+10.1 points), and batch normalisation with global pooling and regularisation (+1.8 and +2.5 points). The gains shrink as accuracy rises, which is why

the error-rate view is informative. Bigger is not better here: the basic CNN has 1.63 million parameters, the final CNN 116 thousand, yet the final CNN is 4.2 points more accurate. The frozen MobileNetV2 (72.58 %) is below even the MLP; fine-tuning adds about 18 points but remains below the custom CNN. The ensemble (97.07 %) does not improve on its best member, so the “best so far” line is flat after step 5.

9.1.3 Per-Class Performance

Of the 43 classes, 6 have an F1 score below 0.90 and 10 are at or above 0.99. Table 9.3 lists the eight weakest. The weakest class is Pedestrians, whose precision is high (0.94) but whose recall is only 0.50: half of the pedestrian signs are classified as something else, for example as Slippery road. Dangerous curve to the right shows the opposite pattern: recall is 1.00 but precision 0.69, because other classes (for example Wild animals crossing) are wrongly assigned to it. The Spearman rank correlation between class size and F1 is +0.28, a weak positive relationship, so class scarcity explains only part of the weakness; visual similarity among warning triangles matters more.

Table 9.3: The eight classes with the lowest F1 on the test set

ID	Class	Precision	Recall	F1	Test images	Training images
27	Pedestrians	0.938	0.500	0.652	60	240
20	Dangerous curve to the right	0.692	1.000	0.818	90	360
21	Double curve	0.867	0.800	0.832	90	330
22	Bumpy road	0.863	0.842	0.852	120	390
30	Beware of ice/snow	0.992	0.800	0.886	150	450
40	Roundabout mandatory	0.842	0.944	0.890	90	360
41	End of no passing	0.932	0.917	0.924	60	240
29	Bicycles crossing	0.865	1.000	0.928	90	270

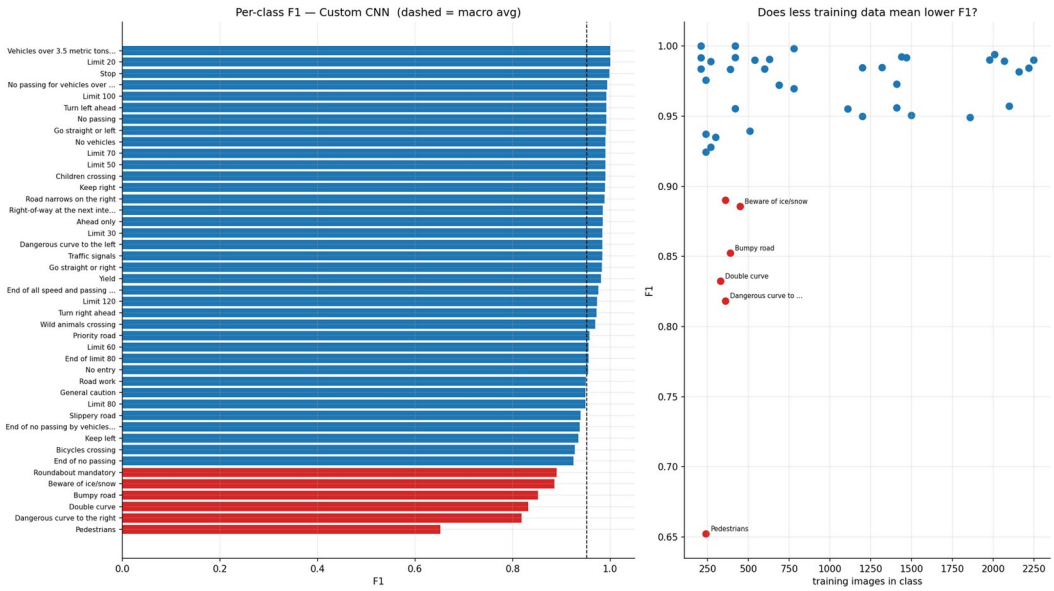


Figure 9.3: F1 score of every class (left) and F1 against class size (right)

9.1.4 Confusion Analysis

The custom CNN makes 384 errors on 12,630 test images. 75 % of them are confusions between signs of the same family (speed limits, prohibitory, warning, mandatory, other), against about 23 % expected by chance. Table 9.4 lists the ten most frequent confusions; the largest is Speed limit 60 km/h predicted as 80 km/h (36 images), a pair that differs in one digit.

Table 9.4: The ten most frequent confusions on the test set

No.	True class	Predicted class	Images
1	Speed limit (60km/h)	Speed limit (80km/h)	36
2	No entry	Priority road	20
3	Bumpy road	Road work	18
4	Speed limit (120km/h)	Speed limit (80km/h)	16
5	Wild animals crossing	Dangerous curve to the right	15
6	Priority road	Yield	14
7	Road work	Bumpy road	12
8	Pedestrians	Slippery road	12
9	Double curve	General caution	11
10	Speed limit (70km/h)	Speed limit (30km/h)	10

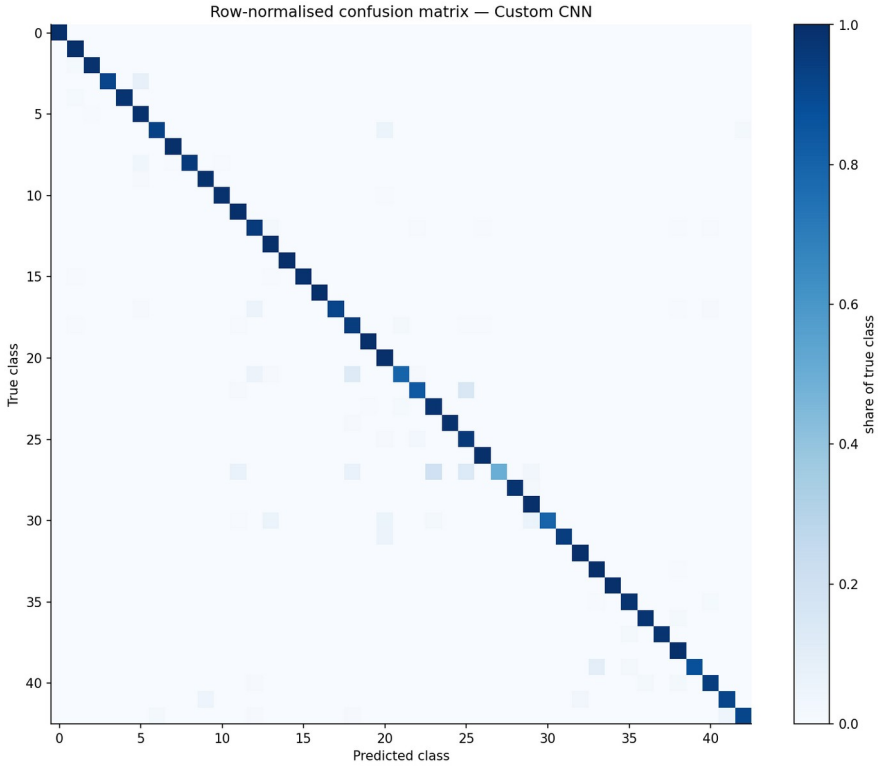


Figure 9.4: Row-normalised confusion matrix of the custom CNN on the test set



Figure 9.5: The twelve most confident errors on the test set

The most confident errors are dominated by very dark or very small images in which the digits of a speed limit are barely visible, so a wrong neighbouring speed limit is chosen with near-certainty.

9.1.5 Explainability

Figure 9.6 shows Grad-CAM maps of the custom CNN and of MobileNetV2 for five correct predictions and five confident errors (notebook version). For correct predictions the CNN's heat lies on the sign's symbol or rim: the arrow, the digits or the corners of a triangle. For the confident errors, which are dark speed-limit signs, the CNN's heat is scattered or sits on a fragment of one digit. MobileNetV2's final feature map is only 4

x 4, so its maps are coarse blocks that often cover the edge of the image or the background even for correct predictions, which is a reason to treat them with caution.

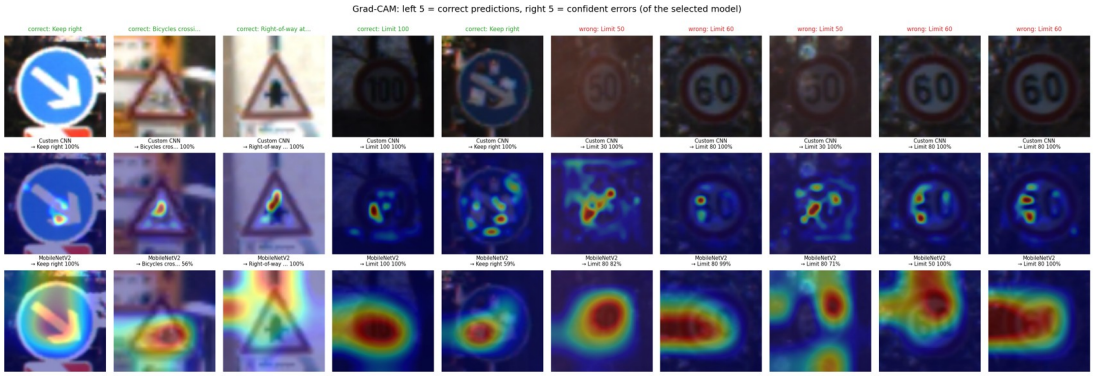


Figure 9.6: Grad-CAM for the custom CNN (middle row) and MobileNetV2 (bottom row): five correct predictions (left) and five confident errors (right)

Figure 9.7 projects the 128-dimensional embedding of 4,000 test images with t-SNE. Images of the same class form tight clusters, and the clusters that touch are those of look-alike signs, in agreement with the confusion analysis.

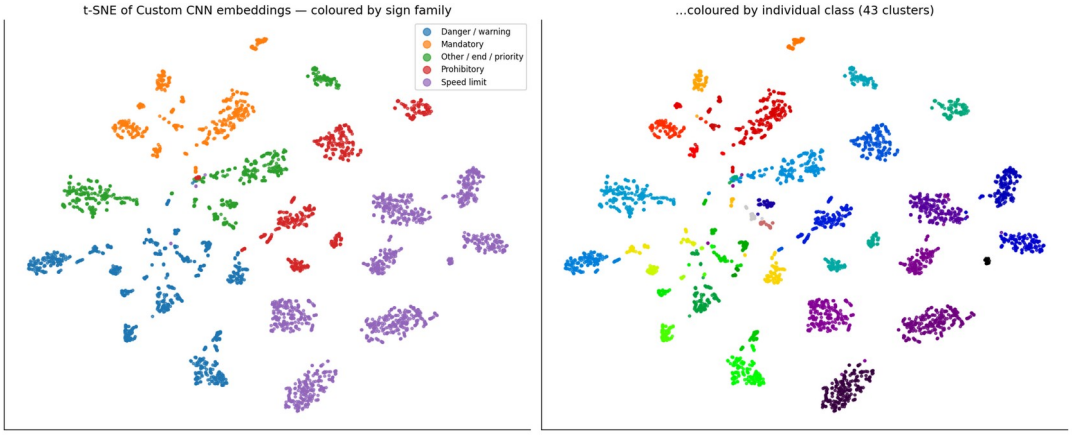


Figure 9.7: t-SNE projection of the penultimate-layer embedding, coloured by sign family and by class

9.1.6 Calibration and Selective Prediction

The expected calibration error of the custom CNN is 0.0105, which is low; the fine-tuned MobileNetV2 is more over-confident (0.0537) and the ensemble is intermediate (0.0267). Wrong predictions are usually made with lower confidence than correct ones, so a threshold can trade coverage for accuracy (Table 9.5, Figure 9.8). With a threshold of 0.866 the model answers 95.5 % of the test images and is right on at least 99.0 % of those; to reach 99.9 % it must defer on 17.4 % of the images. These thresholds are read off the test set and show the trade-off; a deployed system should choose its threshold on separate validation data.

Table 9.5: Selective prediction of the custom CNN on the test set

Target accuracy on answered images	Confidence threshold	Images answered	Images deferred
99.0 %	0.866	95.5 %	4.5 %
99.5 %	0.972	92.0 %	8.0 %
99.9 %	0.999	82.6 %	17.4 %

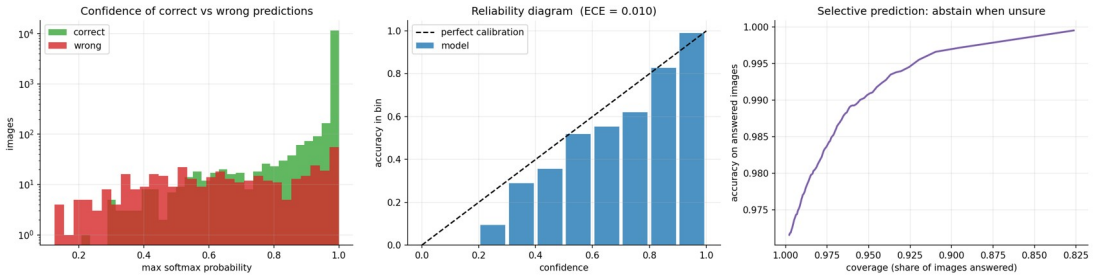


Figure 9.8: Confidence of correct and wrong predictions, reliability diagram and coverage-accuracy curve

9.1.7 Robustness to Corruptions

Table 9.6 and Figure 9.9 show the accuracy of the custom CNN on 6,000 test images (clean accuracy 96.97 %) under six corruptions at three severities. Blur, darkening and glare are tolerated: accuracy stays above 94 % for blur even at the largest kernel and above 82 % for the strongest darkening or glare. The model is fragile to Gaussian noise (62.6 % at σ = 10 and 34.3 % at σ = 25, on a 0 to 255 scale), to occlusion (52.7 % when a fifth of the image is covered, 9.5 % when a third is) and to large rotation (51.6 % at 35 degrees, beyond the roughly 14 degrees used in training). Noise and occlusion were never seen in training, and the rotation result shows a cliff beyond the trained range.

Table 9.6: Accuracy of the custom CNN under corruptions (6,000 test images; clean accuracy 96.97 %)

Corruption	Mild	Medium	Severe	Levels (mild, medium, severe)
Darkening ((brightness))	96.3 %	93.9 %	82.2 %	x0.7, x0.5, x0.3
Glare ((brightness))	94.8 %	91.8 %	84.6 %	x1.4, x1.8, x2.4
Gaussian noise (sigma)	62.6 %	34.3 %	14.1 %	sigma 10, 25, 50
Blur (kernel)	97.1 %	96.6 %	95.0 %	kernel 3, 5, 7
Rotation	96.5 %	93.9 %	51.6 %	10, 20, 35 degrees
Occlusion	82.6 %	52.7 %	9.5 %	10 %, 20 %, 35 % of area

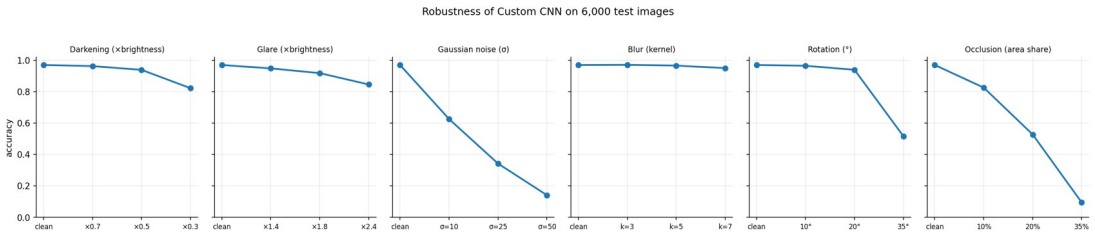


Figure 9.9: Accuracy of the custom CNN under increasing corruption severity

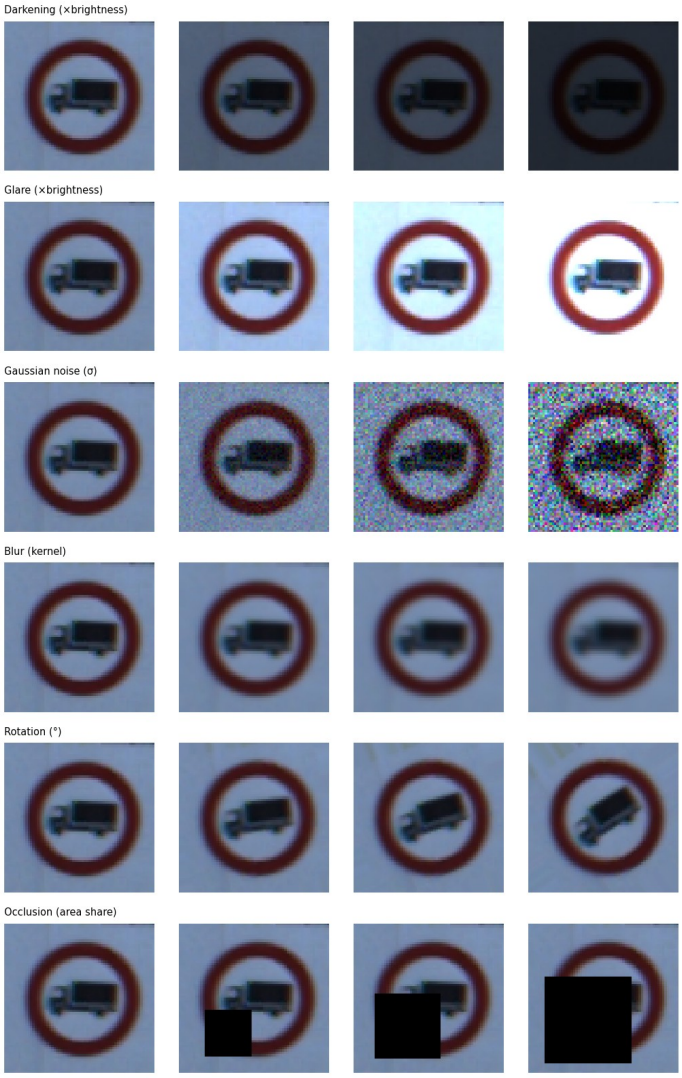


Figure 9.10: Examples of the corruptions at the three severities

9.1.8 Export and Deployment

The selected CNN converts to a TensorFlow Lite file of 0.24 MB (the Keras file is 1.5 MB), which agrees with the Keras model on 100 % of 500 test images, scores 97.6 % on them and runs at 1.24 ms per image on a single CPU thread. The browser implementation loads a 465 KB weight file, and a prediction with Grad-CAM takes about a tenth of a second in Node.js on the development laptop. The whole results

bundle (four models, per-model predictions, tables, figures and a sample of test images) is 86.5 MB, of which the MobileNetV2 and ensemble models make up most.

9.2 Discussion

9.2.1 Merits and Demerits of the Approach

Table 9.7: Merits and demerits of the proposed approach and pipeline

Merits	Demerits
Leakage-free, class-complete validation; test set used once	Handles cropped signs only; needs a detector for real scenes
Compact model: 116,203 parameters, 0.24 MB as TFLite, 18 ms per image in a GPU runtime	Fragile to noise, occlusion and large rotation
Well calibrated (ECE 0.0105); supports abstention with measured trade-off	Over-confident on inputs unlike its training data (for example drawings)
Explained with Grad-CAM and t-SNE; explanation method checked numerically	Grad-CAM maps are coarse and qualitative
Each design step measured; transfer learning compared fairly	Pretrained MobileNetV2 over-fits and does not beat the small CNN at this resolution
Reproducible, with seeds, checksums and assertions	Single seed; run-to-run spread about half a point
Runs in the browser with no server; private; verified against Keras	Only the custom CNN is deployed in the browser; weak classes (Pedestrians recall 0.50)

9.2.2 Comparison with Published Results

The best published GTSRB results are close to or above human accuracy: about 98.8 % for human participants [2] and about 99.5 % for a committee of CNNs [3]. The custom CNN of this study reaches 96.96 %, below both. This is expected from its design: a single small network on 64 x 64 inputs trained for a few minutes, without the input normalisation variants, multi-network committees or long training schedules used in the competition. The aim of the project was not to beat these numbers but to measure honestly, explain and stress a compact model. The result is nevertheless in the range expected for compact CNNs, and its error rate of about 3 % is dominated by identifiable factors (dark speed-limit signs, look-alike warning triangles, a few weak classes) that suggest concrete improvements (Chapter 11).

9.2.3 Why Did the Small CNN Beat MobileNetV2?

Three observations are consistent with the domain-gap explanation proposed in Chapter 3. First, the frozen backbone performs barely better than a linear classifier on raw pixels (72.6 % against 70.2 %, and far below the 83.2 % of the MLP), meaning that ImageNet features at this scale add little. Second, fine-tuning helps substantially (+18 points), meaning that adapting the features matters. Third, the fine-tuned network over-fits (99.8

% training accuracy against 90.6 % validation), so its extra capacity is not constrained by the 33,570 highly correlated training images. None of these observations proves the explanation, and the result should not be generalised: it concerns tiny, low-resolution crops and a frozen or lightly tuned MobileNetV2, not pretrained networks in general. A different resolution, stronger augmentation or a larger unfrozen portion might change the outcome.

9.2.4 Grad-CAM at Saturation

The comparison of two correct implementations of Grad-CAM exposed a weakness of using the class probability as the score. When the network is almost certain, the probability is close to 1 and its derivative with respect to the logits, which is proportional to p(1 - p), is close to zero. The gradients that reach the last convolutional layer are then of the order of floating-point rounding, and after normalisation by the maximum the heat-map is dominated by numerical noise. This happened for 50 of the 116 sample images, and the maps of Keras and of the JavaScript code disagreed on exactly those images, while agreeing to 5.9×10^{-4} on the other 66. Using the class logit instead, as in the original Grad-CAM paper [5], removes the problem: the two implementations then agree to 3.1×10^{-6} on all 116 images. The website therefore uses the logit; the notebook figures in this report use the probability and are most reliable for the less confident cases.

9.2.5 Run-to-Run Variation

The first full run of the pipeline used a validation split with the defect described in Section 7.1.2; the second, reported here, used the corrected split. The two runs share the code, seed and budgets and differ only in the training data of the fitting set and the validation set, so they give a direct, if small, indication of run-to-run variation (Table 9.8).

Table 9.8: Test accuracy in the two full runs

Model	First run	Reported run	Difference
Custom CNN	96.52 %	96.96 %	+0.44
MobileNetV2, fine-tuned	92.35 %	91.26 %	-1.09
Ensemble	97.01 %	96.88 %	-0.13

Differences of about half a point to one point are therefore within the run-to-run spread, and the ordering of the ensemble and the CNN flipped on the test set between the runs.

The large effects (baseline to CNN, frozen to fine-tuned, CNN against fine-tuned MobileNetV2) are many times larger than this spread and held in both runs. Only one seed was used in each run, so a proper confidence interval would require repetition with several seeds.

9.3 Findings

1. **Random validation leaks.** On GTSRB a random split puts 1,291 of 1,307 physical signs on both sides; every validation sign has been seen in training. Grouping by sign removes this, and the grouping must also guarantee class coverage.
2. **A small, well-regularised CNN is enough.** A 116,203-parameter CNN reaches 96.96 % test accuracy and 95.21 % macro-F1 and is better calibrated and about eight times faster than the fine-tuned MobileNetV2.
3. **Each design step matters, and size does not.** The ladder rises monotonically from 5.85 % to 97.61 %; a plain CNN with 14 times more parameters is 4 points worse than the final design.
4. **Transfer learning needs fine-tuning, and here it still falls short.** A frozen backbone reaches 75.16 %, fine-tuning 91.26 %, and the fine-tuned model over-fits.
5. **Errors are between look-alike signs.** 75 % of errors stay inside a sign family; the most frequent is Speed limit 60 predicted as 80. Class size explains only part of the weak classes (Spearman +0.28).
6. **The model can say when it is unsure.** ECE is 0.0105; answering only above a confidence of 0.866 gives 99 % accuracy at 95.5 % coverage.
7. **The weaknesses are noise, occlusion and large rotation.** Accuracy falls to 34 % at noise sigma 25, 53 % with a fifth of the image covered and 52 % at 35 degrees, while blur, darkening and glare are tolerated.
8. **The most confident errors are very dark speed-limit signs.**
9. **Probability-based Grad-CAM is unreliable at saturation.** The class logit should be used as the score.

10. **Compact deployment is practical.** The selected network runs in the browser from a 465 KB file, matches Keras to about 3×10^{-6}, and converts to a 0.24 MB TFLite file with 100 % agreement.

CHAPTER 10: SUMMARY AND CONCLUSION

10.1 Summary of the Work

This project set out to build a traffic-sign classifier for the 43 classes of the German Traffic Sign Recognition Benchmark and, more importantly, to evaluate it in a way that can be trusted. The work began with a verified acquisition of the dataset and an exploratory analysis whose central finding was structural: the 39,209 training images come from only 1,307 physical signs, each photographed about thirty times. A random validation split puts almost all of these signs on both sides of the split. The validation set was therefore built from whole signs, selected per class so that every class is represented, and the code asserts both properties.

Two model families were compared: a compact convolutional network of 116,203 parameters trained from scratch, and MobileNetV2 with ImageNet weights trained in two stages. A ladder of nine steps, from a majority-class baseline to the final network, measured the contribution of each design decision on validation data. The models were then evaluated once on the official test set. The selected custom CNN reached 96.96 % accuracy and 95.21 % macro-F1, ahead of fine-tuned MobileNetV2 (91.26 % and 86.59 %) and an ensemble of both (96.88 % and 95.07 %).

Beyond accuracy, the study examined why and when the model fails. Per-class and confusion analyses showed that errors are mostly between look-alike signs of the same family, and that the weakest classes are Pedestrians, Dangerous curve to the right and Double curve. Grad-CAM and t-SNE showed what the network attends to and how it organises the classes. The network is well calibrated (expected calibration error 0.0105) and can trade coverage for accuracy: answering only above a confidence of 0.866 gives 99 % accuracy on 95.5 % of the images. Corruption tests showed tolerance to blur, darkening and glare and fragility to noise, occlusion and large rotation. A numerical comparison of two implementations showed that Grad-CAM based on the class probability is unreliable for nearly certain predictions.

Finally, the trained network was re-implemented in dependency-free JavaScript, verified against Keras, and published as a static website that classifies images locally, explains them and lets the user stress them. The same pipeline exports every trained model with its results for reuse.

10.2 Conclusions

Table 10.1: Answers to the research questions

Question	Answer
RQ1. How much does a random split overstate performance, and how is the leakage removed?	A random split shares 1,291 of 1,307 signs between training and validation, so every validation sign has been seen. Holding out whole signs, selected per class, removes the leakage and keeps all 43 classes in validation.
RQ2. Small CNN or MobileNetV2, and which decisions matter?	The small CNN is better (96.96 % against 91.26 % test accuracy) and faster. Convolutions, batch normalisation with global pooling, augmentation and dropout each helped; size did not. Fine-tuning gained 16 points over a frozen backbone but over-fitted.
RQ3. Which classes are hardest and why?	Pedestrians (F1 0.65), Dangerous curve to the right (0.82) and Double curve (0.83). Errors are between look-alike signs (75 % within a family); class size explains only part of it (Spearman +0.28).
RQ4. Does the network attend to the sign, and is Grad-CAM reliable?	For correct predictions the heat lies on the symbol or rim. Probability-based Grad-CAM is numerically unreliable for near-certain predictions; the class logit is reliable.
RQ5. Is confidence calibrated, and what does abstention buy?	ECE is 0.0105. A threshold of 0.866 gives 99 % accuracy while answering 95.5 % of images.
RQ6. Which corruptions hurt most?	Noise, occlusion and large rotation. Blur, darkening and glare are tolerated.

The overall conclusion is that, for a task like this, a carefully evaluated small network is a better engineering choice than a larger pretrained one, and that the quality of the evaluation, not only of the model, determines whether a reported accuracy can be believed. The project contributes a reproducible pipeline that makes this evaluation explicit and a verified, serverless demonstration of the resulting model.

10.3 Learning Outcomes

Through this project the team applied machine-learning, software-engineering and communication skills in a single piece of work: designing and running controlled experiments, discovering and correcting a flaw in its own evaluation, implementing a neural network and its explanation method from first principles, verifying numerical equivalence across two implementations, building and deploying an accessible web application, and documenting limitations honestly. These correspond to the program outcomes on engineering knowledge, problem analysis, modern tool usage, ethics, teamwork and communication.

CHAPTER 11: LIMITATIONS OF THE PROJECT AND FUTURE WORK

11.1 Limitations

- **Cropped signs only.** The classifier assumes a tight crop of one sign. Locating signs in a scene needs a detector, which GTSRB cannot train; the repository contains optional detection scripts that require a separately supplied checkpoint and were not evaluated.
- **One country, one dataset.** GTSRB contains German signs. Signs from other countries, newer designs or other weather and camera conditions are untested, and performance measured on one dataset may not transfer to another [28].
- **Single seed.** Each configuration was trained once per run. Two runs of the pipeline differed by up to about one point for some models, so small differences between models are not statistically supported.
- **Resolution.** Images are reduced to 64 x 64 pixels. Details such as the digits of speed limits may be lost, and the most frequent confusion (60 versus 80 km/h) is of this kind.
- **Training did not include noise or occlusion.** The corruption study measures the cost of this omission but the model was not retrained to remove it.
- **Descriptive use of the test set.** The abstention thresholds and the robustness study use test-set behaviour to illustrate trade-offs; they were not used to choose a model, but a deployed threshold should be set on separate validation data.
- **Explanation is qualitative.** Grad-CAM shows influence, not causation, and MobileNetV2's maps are coarse.
- **Timing and hardware.** Latency was measured on a GPU runtime and a laptop with Python or browser overhead, not on automotive hardware.
- **Testing.** There is no automated unit-test suite for the Python package, and the camera input of the website was not exercised.
- **Not safety-certified.** The system is an educational research project. It is not designed, validated or certified for use in vehicles [11], [12].

11.2 Future Work

Table 11.1 lists improvements ordered by how directly the results of this project motivate them.

Table 11.1: Proposed future work and the evidence that motivates it

Priority	Improvement	Motivation from the results
1	Add Gaussian-noise and random-erasing augmentation and widen the rotation range	Accuracy 34 % at noise sigma 25, 53 % with a fifth of the image covered, 52 % at 35 degrees
2	Brightness augmentation and contrast normalisation (for example CLAHE)	Most confident errors are very dark speed-limit signs; strong darkening lowers accuracy to 82 %
3	Higher input resolution (for example 96 pixels) for the CNN	Speed limit 60 versus 80 (36 images), 120 versus 80 (16), 70 versus 30 (10)
4	Class weights or targeted augmentation for the weakest classes	Pedestrians has recall 0.50; Dangerous curve to the right has precision 0.69
5	Stronger regularisation for fine-tuned MobileNetV2 (more augmentation, fewer unfrozen layers, label smoothing)	Training accuracy 99.8 % against 90.6 % validation
6	Repeat every experiment with several seeds and report intervals	Run-to-run spread of up to about one point; ensemble ordering flipped
7	Temperature scaling and a validation-chosen abstention threshold	The threshold was read off test data; ensemble and MobileNetV2 are less calibrated
8	Knowledge distillation from the ensemble into the small CNN [41]	Ensemble has the best top-3 accuracy but costs two models
9	A detection front end for full scenes and video [34]	Classifier handles cropped signs only
10	Out-of-distribution detection and evaluation on signs from other countries	Over-confidence on non-sign inputs; GTSRB covers one country
11	Automated tests: a pytest suite for the package and browser tests for the website	Testing limitations (Section 8.6)

11.3 Ethical and Safety Considerations

Traffic-sign recognition is a safety-relevant technology, and over-stating the reliability of a classifier can cause harm. For this reason the website states that it handles cropped signs only, is not a driving system and can be over-confident on inputs unlike its training data; the report separates decisions made on validation data from descriptive analyses that use the test set; and the limitations above are stated explicitly. Images supplied to the website are processed locally and are never uploaded. The sample images shown come from a public research benchmark and are used for illustration with attribution; their redistribution should follow the dataset's terms.

REFERENCES

[1] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: WHO, 2023.

[2] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural Netw.*, vol. 32, pp. 323–332, Aug. 2012.

[3] D. Cireşan, U. Meier, J. Masci, and J. Schmidhuber, "Multi-column deep neural network for traffic sign classification," *Neural Netw.*, vol. 32, pp. 333–338, Aug. 2012.

[4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, Dec. 2012.

[5] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 618–626.

[6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, Sydney, Australia, 2017, pp. 1321–1330.

[7] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in NIPS*, vol. 30, 2017, pp. 4878–4887.

[8] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. ICLR*, New Orleans, LA, USA, 2019.

[9] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing, and C. Igel, "Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark," in *Proc. IJCNN*, Dallas, TX, USA, 2013.

[10] Z. Zhu, D. Liang, S. Zhang, X. Huang, B. Li, and S. Hu, "Traffic-sign detection and classification in the wild," in *Proc. CVPR*, Las Vegas, NV, USA, 2016, pp. 2110–2118.

[11] Road Vehicles — Functional Safety, ISO Standard 26262, International Organization for Standardization, Geneva, Switzerland, 2018.

[12] Road Vehicles — Safety of the Intended Functionality, ISO Standard 21448, International Organization for Standardization, Geneva, Switzerland, 2022.

[13] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The German Traffic Sign Recognition Benchmark: A multi-class classification competition," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, San Jose, CA, USA, 2011, pp. 1453–1460.

[14] P. Sermanet and Y. LeCun, "Traffic sign recognition with multi-scale convolutional networks," in *Proc. IJCNN*, San Jose, CA, USA, 2011, pp. 2809–2813.

[15] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, 2012, pp. 1097–1105.

[17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[18] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. ICML*, 2010, pp. 807–814.

[19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.

[21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, 2015.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[23] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[24] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in NIPS*, vol. 27, 2014, pp. 3320–3328.

[25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. CVPR*, Miami, FL, USA, 2009, pp. 248–255.

[26] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. CVPR*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.

[28] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. CVPR*, Colorado Springs, CO, USA, 2011, pp. 1521–1528.

[29] scikit-learn developers, "sklearn.model_selection.GroupShuffleSplit," scikit-learn documentation. [Online]. Available: https://scikit-learn.org. Accessed: Oct. 2026.

[30] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[31] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.* vol. 9, pp. 2579–2605, Nov. 2008.

[32] C. Szegedy *et al.*, "Intriguing properties of neural networks," in *Proc. ICLR*, Banff, Canada, 2014.

[33] K. Eykholt *et al.*, "Robust physical-world attacks on deep learning visual classification," in *Proc. CVPR*, Salt Lake City, UT, USA, 2018, pp. 1625–1634.

[34] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. CVPR*, Las Vegas, NV, USA, 2016, pp. 779–788.

[35] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, Savannah, GA, USA, 2016, pp. 265–283.

[36] F. Chollet *et al.*, "Keras," 2015. [Online]. Available: https://keras.io. Accessed: Oct. 2026.

[37] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020.

[38] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[39] Google, "Google Colaboratory." [Online]. Available: https://colab.research.google.com. Accessed: Oct. 2026.

[40] "GTSRB: German Traffic Sign Recognition Benchmark (dataset archive)," Zenodo, record 13741936. [Online]. Available: https://zenodo.org/records/13741936. Accessed: Oct. 2026.

[41] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

Appendix A: Source Code of Selected Modules (Annexure 1)

The complete source code is available in the project repository (https://github.com/BriskAM/mait-mini). This appendix reproduces selected modules that complement the listings of Chapter 7.

Listing A.1: Dataset download with streaming MD5 verification (notebook)

```
def md5_of(path, chunk=1 << 20):
    h = hashlib.md5()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(chunk), b""): h.update(block)
    return h.hexdigest()


if not archive.exists() or md5_of(archive) != EXPECTED_MD5:
    urllib.request.urlretrieve(URL, archive, hook)          # progress hook prints every
        ~25 MB
got = md5_of(archive)
assert got == EXPECTED_MD5, "Checksum mismatch: delete data.zip and rerun."
with zipfile.ZipFile(archive) as z: z.extractall(WORK / "gtsrb")
```

Listing A.2: Training helper with early stopping and learning-rate reduction (notebook)

```
def fit(model, epochs, lr, patience, tag):
    model.compile(optimizer=keras.optimizers.Adam(lr),
        loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    cbs = [keras.callbacks.EarlyStopping(monitor="val_accuracy", mode="max",
        patience=patience, restore_best_weights=True),
           keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.4,
               patience=max(1, patience // 2), min_lr=1e-6)]
    t0 = time.time()
    h = model.fit(X_fit, y_fit, validation_data=(X_val, y_val), batch_size=BATCH,
        epochs=epochs, callbacks=cbs, verbose=2)
    print(f"[{tag}] {len(h.history['loss'])} epochs in {time.time() - t0:.0f}s | best val
        acc {max(h.history['val_accuracy']):.4f}")
    return h.history, time.time() - t0
```

Listing A.3: Scoring: accuracy, macro-F1, top-3 accuracy and expected calibration error (notebook)

```
def ece_score(conf, correct, bins=15):
    edges = np.linspace(0, 1, bins + 1); e = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any(): e += m.mean() * abs(correct[m].mean() - conf[m].mean())
    return e


def score_from_probs(p, y):
    pred = p.argmax(1); conf = p.max(1)
    return dict(probs=p, pred=pred, conf=conf, accuracy=(pred == y).mean(),
                macro_f1=f1_score(y, pred, average="macro", labels=range(NUM_CLASSES),
                    zero_division=0),
                top3=np.mean([yy in np.argsort(pp)[-3:] for yy, pp in zip(y, p)]),
                    ece=ece_score(conf, pred == y))
```

Listing A.4: Grad-CAM for a flat model (Python, notebook version)

```
def gradcam(model, x_uint8, target=None):
    kind, cam_model, idx = _cam_parts(model); x =
        tf.convert_to_tensor(x_uint8[None].astype("float32"))
    with tf.GradientTape() as tape:
        conv_out, probs = cam_model(x, training=False)       # last conv output and
            network output
        cls = int(tf.argmax(probs[0])) if target is None else target
        score = probs[:, cls]                                 # post-softmax score (see
            Section 9.2.4)
    grads = tape.gradient(score, conv_out)
    cam = tf.nn.relu(tf.reduce_sum(conv_out * tf.reduce_mean(grads, axis=(1, 2))[:, None,
        None, :], axis=-1))[0].numpy()
    cam = cam / (cam.max() + 1e-8)
    return cv2.resize(cam, (64, 64), interpolation=cv2.INTER_CUBIC).clip(0, 1), cls
```

Listing A.5: Convolution and max pooling in JavaScript (nn.js)

```
function conv3x3(inp, H, W, Cin, w, b, Cout) {
```

```
  const out = new Float32Array(H * W * Cout);
  for (let y = 0; y < H; y++) for (let x = 0; x < W; x++) {
    const o = (y * W + x) * Cout;
    for (let co = 0; co < Cout; co++) out[o + co] = b[co];
    for (let ky = 0; ky < 3; ky++) { const yy = y + ky - 1; if (yy < 0 || yy >= H)
        continue;
      for (let kx = 0; kx < 3; kx++) { const xx = x + kx - 1; if (xx < 0 || xx >= W)
          continue;
        const ib = (yy * W + xx) * Cin, wb = (ky * 3 + kx) * Cin * Cout;
        for (let ci = 0; ci < Cin; ci++) { const v = inp[ib + ci], wo = wb + ci * Cout;
          for (let co = 0; co < Cout; co++) out[o + co] += v * w[wo + co]; } } }
    for (let co = 0; co < Cout; co++) if (out[o + co] < 0) out[o + co] = 0;      // ReLU
  }
  return out;
}
```

Listing A.6: Exporting the Keras weights for the browser (build_pages_data.py, abridged)

```
order = [("conv1", "conv2d", 2), ("bn1", "batch_normalization", 4), ("conv2", "conv2d_1",
    2), ("bn2", "batch_normalization_1", 4),
         ("conv3", "conv2d_2", 2), ("bn3", "batch_normalization_2", 4), ("dense1",
             "dense", 2), ("dense2", "dense_1", 2)]
for tag, group, n in order:
    for i in range(n):
        arr = np.asarray(h5[f"layers/{group}/vars/{i}"], dtype="<f4")
        entries.append({"name": f"{tag}.{names[key][i]}", "shape": list(arr.shape),
            "offset": offset})
        chunks.append(arr.ravel()); offset += arr.size
blob = np.concatenate(chunks).astype("<f4")       # 116,203 float32 values = 465 KB
(out / "data" / "cnn_weights.bin").write_bytes(blob.tobytes())
```

Appendix B: Per-Class Results of the Custom CNN

Table B.1 gives the precision, recall and F1 score of every class for the selected custom CNN on the official test set, together with the number of test and training images.

Table B.1: Per-class test results of the custom CNN

ID	Class	Precision	Recall	F1	Test	Train
0	Speed limit (20km/h)	1.000	1.000	1.000	60	210
1	Speed limit (30km/h)	0.970	0.999	0.984	720	2220
2	Speed limit (50km/h)	0.993	0.987	0.990	750	2250
3	Speed limit (60km/h)	1.000	0.916	0.956	450	1410
4	Speed limit (70km/h)	1.000	0.980	0.990	660	1980
5	Speed limit (80km/h)	0.911	0.990	0.949	630	1860
6	End of speed limit (80km/h)	0.986	0.927	0.955	150	420
7	Speed limit (100km/h)	0.987	0.998	0.992	450	1440
8	Speed limit (120km/h)	0.993	0.953	0.973	450	1410
9	No passing	0.992	0.992	0.992	480	1470
10	No passing for vehicles over 3.5 metric tons	0.997	0.991	0.994	660	2010
11	Right-of-way at the next intersection	0.977	0.993	0.985	420	1320
12	Priority road	0.961	0.954	0.957	690	2100
13	Yield	0.964	1.000	0.982	720	2160
14	Stop	1.000	0.996	0.998	270	780
15	No vehicles	0.990	0.990	0.990	210	630
16	Vehicles over 3.5 metric tons prohibited	1.000	1.000	1.000	150	420
17	No entry	0.997	0.917	0.955	360	1110
18	General caution	0.953	0.946	0.950	390	1200
19	Dangerous curve to the left	0.968	1.000	0.984	60	210
20	Dangerous curve to the right	0.692	1.000	0.818	90	360
21	Double curve	0.867	0.800	0.832	90	330
22	Bumpy road	0.863	0.842	0.852	120	390
23	Slippery road	0.902	0.980	0.939	150	510
24	Road narrows on the right	0.989	0.989	0.989	90	270
25	Road work	0.941	0.960	0.951	480	1500
26	Traffic signals	0.968	1.000	0.984	180	600
27	Pedestrians	0.938	0.500	0.652	60	240
28	Children crossing	1.000	0.980	0.990	150	540
29	Bicycles crossing	0.865	1.000	0.928	90	270
30	Beware of ice/snow	0.992	0.800	0.886	150	450
31	Wild animals crossing	0.996	0.944	0.970	270	780
32	End of all speed and passing limits	0.952	1.000	0.976	60	240
33	Turn right ahead	0.950	0.995	0.972	210	689
34	Turn left ahead	0.984	1.000	0.992	120	420

ID	Class	Precision	Recall	F1	Test	Train
35	Ahead only	0.992	0.977	0.984	390	1200
36	Go straight or right	0.983	0.983	0.983	120	390
37	Go straight or left	1.000	0.983	0.992	60	210
38	Keep right	0.984	0.994	0.989	690	2070
39	Keep left	1.000	0.878	0.935	90	300
40	Roundabout mandatory	0.842	0.944	0.890	90	360
41	End of no passing	0.932	0.917	0.924	60	240
42	End of no passing by vehicles over 3.5 metric tons	0.965	0.911	0.937	90	240

Appendix C: Reproducing the Work

C.1 Running the Experiments

1. Open the notebook `notebooks/traffic_sign_recognition_demo.ipynb` in Google Colab (the badge in the repository README opens it directly).
2. Choose *Runtime, Change runtime type, GPU*.
3. In the first code cell set `MODE` to `smoke` (2-minute sanity run), `demo` or `full` (reported results).
4. Run all cells. The notebook downloads and verifies the dataset, trains and evaluates the models, and finally downloads `gtsrb_demo_results.zip` with the models, results and figures.

C.2 Rebuilding the Website Data

1. Unzip the results bundle to a folder.
2. Run `python scripts/build_pages_data.py --results <folder> --data data/raw/gtsrb --out site` (needs numpy, pandas, Pillow and h5py).
3. Serve the site locally with `python3 -m http.server 8000 --directory site` and open http://localhost:8000.
4. Verify the browser network against Keras with `node scripts/verify_pages_model.mjs` (using reference outputs generated from the same weights).
5. Push to the `main` branch; the workflow `.github/workflows/pages.yml` publishes the site to GitHub Pages.

C.3 Repository Layout

Table C.1: Layout of the project repository

Folder or file	Contents
`notebooks/`	The demonstration notebook (generated by `scripts/build_demo_notebook.py`) and an earlier lighter notebook
`src/traffic_sign_recognition/`	Reusable Python package: data, models, training, evaluation, Grad-CAM, inference

Folder or file	Contents
`scripts/`	Dataset download, analysis and document builders; website data build; browser-network verification
`site/`	The static website: page, styles, JavaScript modules, generated data and figures
`docs/`	Earlier synopsis and report, literature survey, presenter documentation
`report/`	Sources that generate this report (diagrams, content, build script)

Appendix D: Research and Publication Status (Annexure 2)

As required by the departmental notice, the status of research output related to this project is as follows.

Table D.1: Research and publication status

Item	Status
Book chapter	None submitted, accepted or published
Conference paper	None submitted, accepted or published
Research article	None submitted, accepted or published
Hard copy of research paper	Not applicable

The experimental results of this project (the leakage analysis, the progress ladder and the reliability study) could form the basis of a short paper; this is listed as a possible follow-up.